Базы данных
Рубежный контроль №2

Теория

Вопрос 1

1. Системные базы данных: состав и назначение. Именование объектов базы данных. Схемы.

база данных **master**	главная база данных, содержит всю системную информацию СУБД SQL Server (общие для всего экземпляра метаданные, такие как сведения об учетных записях, связанных серверах, и параметры конфигурации системы), а также информацию обо всех остальных базах данных
база данных **Resource**	доступна только для чтения, содержит все входящие в SQL Server системные объекты (такие как **sys.objects**), которые физически расположены в базе данных **Resource**, а логически отображаются для каждой базы данных в схеме **sys**
база данных **msdb**	используется агентом SQL Server (SQL Server Agent) для создания расписания предупреждений и заданий, а также другими компонентами (SQL Server Management Studio, Service Broker and Database Mail)
база данных **model**	используется в качестве шаблона для всех баз данных, созданных для экземпляра SQL Server (в том числе **tempdb**)
база данных **tempdb**	хранит все служебные и временные объекты (например, временные таблицы, временные хранимые процедуры и результаты промежуточных вычислений, создаваемые как пользователями, так и службами SQL Server; повторно создается при каждом запуске SQL Server, временные объекты удаляются автоматически при отключении.

Идентификатор — имя сервера, базы данных и её объектов (таблиц, представлений, столбцов, индексов, триггеров, процедур, ограничений и правил, и т.п.), создаваемое при определении объекта и используемое для обращения к нему.

Классы идентификаторов:

- обычные: соответствуют правилам форматирования идентификаторов:
  - первый символ — буква (Unicode Standard 3.2) или символы _, @ или #;
  - последующие символы — буквы, десятичные цифры или символы _, @, # или $;
  - не должны совпадать с зарезервированными словами (независимо от регистра);
  - не должны содержать пробелы и спецсимволы;
- расширенные (идентификаторы с разделителем):

  - могут содержать любые символы;
  - заключаются в квадратные скобки или двойные кавычки: [..], “..”
- длина идентфикаторов — до 128 символов.

Идентификаторы с разделителями:

```
SET QUOTED_IDENTIFIER OFF -- default
GO
CREATE TABLE [table] (
  tablename char(128) NOT NULL,
  [USER] char(128) NOT NULL,
  [SELECT] char(128) NOT NULL,
  [INSERT] char(128) NOT NULL,
  [UPDATE] char(128) NOT NULL,
  [DELETE] char(128) NOT NULL
);
GO

SET QUOTED_IDENTIFIER ON;
GO
SELECT * FROM "table";
GO

SET QUOTED_IDENTIFIER OFF;
GO
SELECT * FROM [table];
GO
```

Виды идентификаторов:

- имена объектов: `[[[server.][database].][schema_name].]object_name[.column_name]`
- локальные переменные и параметры: `@...`
- локальные временные таблицы и процедуры: `#...`
- глобальные временные объекты (таблицы и процедуры): `##...`

Схема — это коллекция объектов базы данных, формирующая единое пространство имён (набор, в котором у каждого элемента есть своё уникальное имя). Дополнительно схема позволяет администрировать права доступа к содержащимся в ней объектам.

```
CREATE SCHEMA Sprockets
  CREATE TABLE NineProngs (
    source     int,
    cost       int,
    partnumber int) ;

ALTER SCHEMA HumanResources TRANSFER Person.Address;

DROP TABLE Sprockets.NineProngs;
DROP SCHEMA Sprockets;
```

2. Физическая структура базы данных: файлы и файловые группы.

Файлы данных:

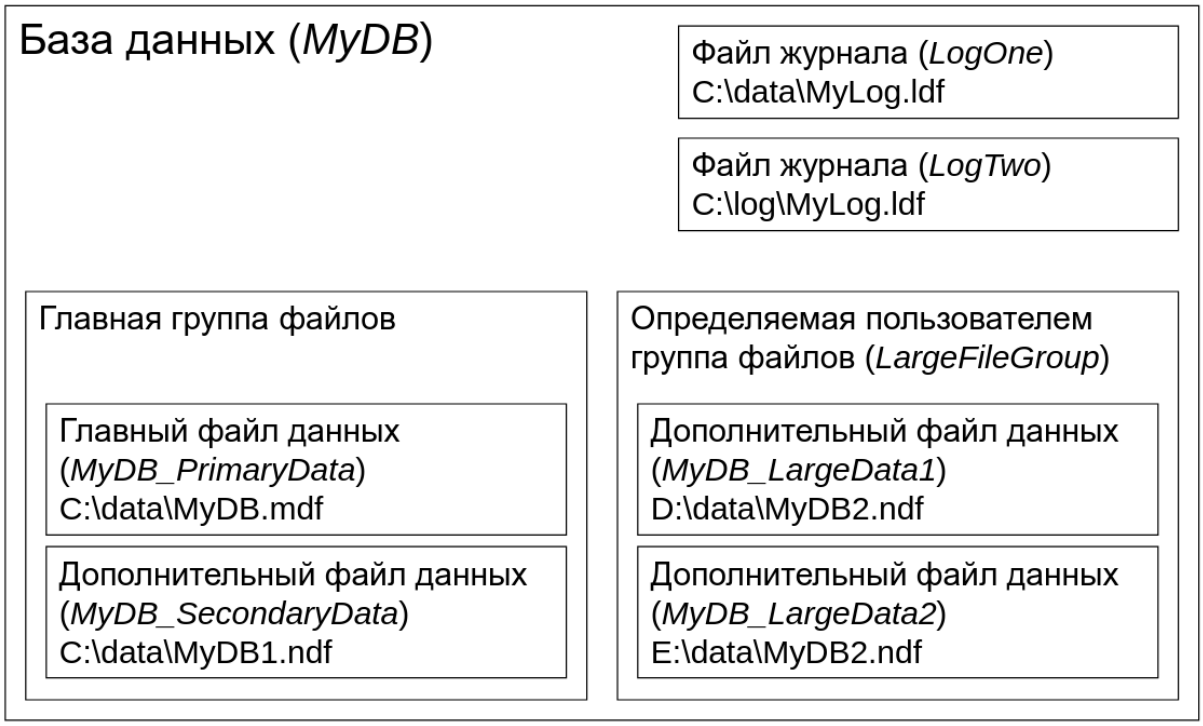
первичный (*.mdf)	содержит сведения, необходимые для запуска базы данных, и ссылки на другие файлы в базе данных; данные и объекты пользователя могут храниться в данном файле или во вторичном файле данных; в каждой базе данных имеется один первичный файл данных;
вторичный (.ndf)	не являются обязательными; могут быть использовы для распределения данных на несколько дисков, в этом случае каждый файл записывается на отдельные диск;
журнал транзакций (.ldf)	содержат сведения, используемые для восстановаления базы данных; для каждой базы данных должен существовать хотя бы один файл журнала.

Файловые группы

Файлы данных могут быть объединены в файловые группы для удобства распределения и администрирования.

первичная	первичная файловая группа, содержащая первичный файл все системные таблицы размещены в первичной файловой группе;
пользовательская	любая файловая группа, созданная пользователем при создании или при изменении базы данных;
файловые группы по умолчанию	если в базе данных создаются объекты без указания файловой группы, к которой они относятся, они назначаются файловой группе по умолчанию; только одна файловая группа создаётся как файловая группа по умолчанию; первичная файловая группа является файловой группой по умолчанию.

Пример физической структуры базы данных:



3. Основные элементы синтаксиса Transact-SQL: константы, встроенные функции, выражения.

Константы

- Числовые (bit, integer, decimal, float, real, money, uniqueidentifier, константы двоичных строк)
- Строковые (заключены в одинарные кавычки; строки Unicode используют префикс N; одинарные кавычки, содержащиеся в строке, дублируются; если SET QUOTED_IDENTIFIER OFF, можно использовать двойные кавычки)
- Константы даты (формат ISO 8601 - ‘yyyy-mm-ddThh:mm:ss[.mmm]’ ; алфавитный формат; числовой формат; строковый формат без разделителей (6- и 8-разрядные строки))

Встроенные функции можно использовать и включать в следующие конструкции:

- список выбора инструкции SELECT;
- предложение WHERE инструкций SELECT, INSERT, DELETE или UPDATE для ограничения количества строк, удовлетворяющих условиям запроса;
- условие поиска в предложении WHERE представления, которое должно динамически соответствовать пользователю или среде во время выполнения;
- любое выражение;
- ограничение CHECK или триггер для поиска указанных значений при вставке данных;
- ограничение DEFAULT или триггер для вставки значений по умолчанию;

Примеры категорий функций:

- статические функции (AVG, MIN, SUM, COUNT, MAX)

- Функции для работы с курсорами (@@CURSOR_ROWS, CURSOR_STATUS, @@FETCH_STATUS)
- Математические функции (ABS, RAND, COS, EXP, ROUND)
- Функции метаданных (DB_NAME, OBJECT_NAME,)
- Строковые функции (STR, REPLACE, SUBSTRING, LEN, LOWER,)
- Системные функции (ISDATE, ISNULL, NEWID, CAST, CONVERT,…)

Выражение - это сочетание идентификаторов, значений и операторов, которое SQL Server 2025 может вычислить для получения результата;

Выражение может быть: константой; функцией; именем столбца; переменной; вложенным запросом; функцией CASE, NULLIF или COALESCE; выражение также может быть построено из комбинаций этих сущностей, соединенных операторами Выражения могут использоваться в различных контекстах во время доступа к данным или их изменения, например: как часть данных, получаемых в запросе; В качестве условий поиска данных, отвечающих определенному набору критериев;

4. Типы данных: атрибуты, категории, применение. Приведение типов.

Типы данных

- С объектом, содержащим данные, связан тип, определяющий виды данных, которые могут храниться в объекте;
- типы данных имеют следующие объекты:
  - столбцы таблиц и представлений;
  - параметры хранимых процедур / функций;
  - переменные;
  - функции Transact-SQL, возвращающие одно или несколько значений;
  - хранимые процедуры, возвращающие значение (всегда имеет тип integer);
- можно создавать два вида пользовательских типов данных:
  - типы данных псевдонима создаются на основе базовых типов данных (позволяют назначить типу данных имя, лучше характеризующее типы значений, которые будут храниться в объекте): CREATE TYPE SSN FROM VARCHAR(11) NOT NULL;
  - пользовательские типы данных CLR основаны на типах данных, созданных в управляемом коде и помещенных в сборку SQL Server.

Атрибуты типов данных

- при назначении типа данных объекту определяются четыре атрибута объекта:
  - вид данных, содержащихся в объекте;
  - размер или длина хранимого объектом значения: количество байт, используемых для хранения числа;
  - точность: количество десятичных знаков;

    - масштаб числа: количество десятичных знаков справа от десятичного разделителя;
- точность (precision) и масштаб (scale) числовых типов данных, кроме decimal и numeric, фиксированы.

Встроенные типы данных

- точные числовые типы:
    - tinyint, smallint, int, bigint;
    - bit;
    - decimal, numeric;
    - money, smallmoney;
- вещественные типы:
    - float, real;
- дата и время:
    - date, time, datetime, datetime2, smalldatetime, datetimeoffset;
- строковые типы:
    - char(n), varchar(n), text;
    - Unicode: nchar(n), nvarchar(n), ntext;
- двоичные строки:
    - binary(n), varbinary(n), image;
- специальные типы:
    - geography, geometry - пространственные типы (spatial types);
    - hierarchyid;
    - json, xml;
    - vector;
    - rowversion;
    - sql_variant;
    - cursor;
    - table;
    - uniqueidentifier.

Преобразование типов данных:

- тип выражения определяется типом входящих в него операндов;
- если, оператор соединяет два выражения разных типов, то производится приведение значений левой и правой частей к типу с высшим приоритетом согласно правилам предшествования типов (*data type precedence*) — по мере убывания приоритета: пользовательские типы, json, sql_variant, xml, datetimeoffset, datetime2, datetime, smalldatetime, date, time, float, real, decimal, money, smallmoney, bigint, int, smallint, tinyint, bit, ntext, text, image, timestamp, uniqueidentifier, nvarchar (включая nvarchar(max)), nchar, varchar (включая varchar(max)), char, varbinary (включая varbinary(max)), binary;
- если неявное приведение типов невозможно, выдаётся сообщение об ошибке;
- явное приведение типов:
    - `CAST ( expression AS data_type [ ( length ) ] )`

  - `CONVERT ( data_type [ ( length ) ] , expression [ , style ])`
- очевидно, что в ряде случаев может быть невозможно как неявное, так и явное приведение типов.

5. Категории целостности данных.

Целостность данных

- целостность данных (data integrity) – это гарантия точности, полноты и согласованности данных организации на любом этапе их жизненного цикла;
- цель обеспечения целостности данных:
  - гарантировать, что аналитика данных основана на достоверной информации;
  - гарантировать соответствие отраслевым стандартам и законодательным ограничениям;
- комплексный подход, затрагивающий технологическую инфраструктуру организации, политики и лиц, работающих с данными:
  - проверка на наличие ошибок;
  - процедуры валидации;
  - защита данных организации от потерь, утечек и искажений (шифрование, контроль доступа, резервное копирование);
- в базах данных обеспечивается, в том числе, ограничениями целостности данных (integrity constraints).

Категории целостности данных

- *сущностная целостность* (entity integrity): определяет уникальность сущности в рамках хранимого набора данных;
- *доменная целостность* (domain integrity): определяет достоверность значений в конкретном элементе данных (количество, формат и допустимые для ввода значения);
- *ссылочная целостность* (referential integrity): обеспечивает наличие соответствующих взаимосвязей в связанных элементах данных, предотвращая появление потерянных (orphaned) данных при добавлении, изменении или удалении;
- *пользовательская целостность* (user-defined integrity): определяет правила и ограничения для данных в соответствии с требованиями предметной области (бизнес-правила);
- *физическая целостность* (physical integrity): обеспечивает целостность данных при их хранении и извлечении.

Сущностная целостность:

- определяет уникальность сущности в рамках хранимого набора данных, т.е. уникальность строки в конкретной таблице;
- предполагает обеспечение целостности столбцов идентификаторов или первичного ключа таблицы с помощью:
  - ограничений PRIMARY KEY;

- ограничений UNIQUE;
- индексов;
- других средств, специфичных для конкретной СУБД.

Доменная целостность:

- определяет достоверность записей в конкретном столбце (формат и допустимые для ввода значения);
- доменная целостность обеспечивается:
  - определением типа данных: с любым объектов, содержащим данные (столбцы таблиц и представлений, переменные, параметры и результаты функций), связан тип, определяющий виды данных, которые могут храниться в этом объекте;
  - определение формата при помощи ограничений CHECK;
  - определением набора или диапазона возможных значений при помощи ограничений FOREIGN KEY, CHECK, DEFAULT, NOT NULL.

Ссылочная целостность:

- обеспечивает согласованность записей в связанных таблицах — предотвращаются следующие действия:
  - добавление или изменение записей в связанной таблице, если в первичной таблице нет соответствующей записи;
  - изменение значений в первичной таблице, которое приводит к появлению потерянных записей в связанной таблице;
  - удаление записей из первичной таблицы, если имеются совпадающие ключи в дочерних таблицах;
- ссылочная целостность, как правило, основана на связи первичных и внешних ключей и обеспечивается с помощью ограничений FOREIGN KEY.

Пользовательская целостность:

- определяет правила и ограничения для данных в соответствии с требованиями предметной области (бизнес-правила), дополняющие ограничения других категорий целостности;
- пользовательская целостность обеспечивается:
  - ограничениями уровня столбца и уровня таблицы (в инструкции CREATE TABLE);
  - средствами программирования, специфичными для конкретной СУБД (триггерами, функциями, хранимыми процедурами и т.п.).

6. Создание, изменение и удаление таблиц: реализация сущностной целостности, вычисляемые столбцы.

Минимальный синтаксис инструкции CREATE TABLE требует указания имени новой таблицы, а также имен и типов данных каждого столбца (имена таблиц и столбцов должны соответствовать синтаксису иденитификаторов)

```
CREATE TABLE products (
  product_no integer,
  name nvarchar(max),
  price numeric
) ON [PRIMARY];

ALTER TABLE doc_exc
  ADD column_b VARCHAR(100) NOT NULL
    CONSTRAINT exb_unique UNIQUE;

DROP TABLE Orders
```

Вычисляемые столбцы

- вычисляемый столбец – столбец, вычисляемый из других столбцов таблицы (не допускаются подзапросы);
- хранимый вычисляемый столбец вычисляется при записи (добавлении или изменении) и занимает место в таблице так же, как и обычный столбец.
- произвести запись непосредственно в генерируемый столбец нельзя.

```
CREATE TABLE dbo.tableX
(
low INT,
high INT,
average AS (low + high) / 2,
delta AS (low + high) PERSISTED
);
```

Ограничение первичного ключа:

- ограничение первичного ключа подразумевает, что образующий его столбец или набор столбцов может быть уникальным идентфикатором строк в таблице;
- значения входящих в первичный ключ столбцов должны быть уникальными в совокупности и каждое из них должно быть отлично от NULL.

```
CREATE TABLE dbo.Employee
(
  EmployeeID INT PRIMARY KEY CLUSTERED
);

CREATE TABLE [dbo].[Supply](
  [PartID] [int] NOT NULL,
  [SupplierID] [int] NOT NULL,
  [Price] [float] NOT NULL,
  [DeliveryTime] [int] NOT NULL,
  CONSTRAINT [PK_Supply] PRIMARY KEY CLUSTERED ([PartID],[SuplierID])
);
```

Ограничение UNIQUE:

- ограничение уникальности гарантирует, что данные в определённом столбце или группе столбцов уникальны среди всех строк таблицы;

- допускается наличие неопределённых значений (NULL) в столбцах, входящих в ограничение;
- допускается наличие нескольких ограничений UNIQUE в таблице.

```
CREATE TABLE Production.TransactionHistoryArchive4
(
  TransactionID int NOT NULL,
  CONSTRAINT AK_TransactionID UNIQUE(TransactionID)
);

ALTER TABLE Person.Password
ADD CONSTRAINT AK_Password UNIQUE (PasswordHash, PasswordSalt);

ALTER TABLE dbo.DocExc
DROP CONSTRAINT UNQ_ColumnB_DocExc;
```

7. Создание, изменение и удаление таблиц: формирование значений суррогатных ключей.

Минимальный синтаксис инструкции CREATE TABLE требует указания имени новой таблицы, а также имен и типов данных каждого столбца (имена таблиц и столбцов должны соответствовать синтаксису иденитификаторов)

```
CREATE TABLE products (
  product_no integer,
  name nvarchar(max),
  price numeric
) ON [PRIMARY];

ALTER TABLE doc_exc
  ADD column_b VARCHAR(100) NOT NULL
    CONSTRAINT exb_unique UNIQUE;

DROP TABLE Orders
```

8. Создание, изменение и удаление таблиц: реализация доменной целостности, значения по умолчанию.

Ограничение CHECK

- ограничение CHECK – некоторое логическое выражение, вычисляемое при вставке или обновлении записи таблицы для проверки соответствия данных рассматриваемому ограничению;
- ограничение CHECK может быть определено:
  - на уровне отдельного столбца;
  - на уровне таблицы.

```
CREATE TABLE products (
product_no integer,
name nvarchar (255),
price numeric CHECK (price > 0),
discounted_price numeric,
```

```
CHECK (discounted_price > 0),
CONSTRAINT valid_discount CHECK (price > discounted_price) );
```

Ограничение NOT NULL

- ограничение NOT NULL для столбца указывает, что столбец не может принимать неопределённое значение (NULL);
- данное ограничение явно задаётся в определении столбца, при этом также можно присвоить ему имя.

Значения по умолчанию

- значение по умолчанию – значение, которое будет автоматически присвоено столбцу, если оно не будет явно указано при добавлении новой записи;
- если значение по умолчанию не объявлено явно, им считается значение NULL;
- значение по умолчанию может быть выражением, которое в этом случае вычисляется в момент присваивания значения по умолчанию.

9. Создание, изменение и удаление таблиц: реализация ссылочной целостности, действия для ограничений внешнего ключа.

Ограничение внешнего ключа:

- ограничение внешнего ключа указывает, что значения столбца (или группы столбцов) должны соотвествовать значениям в некоторой строке другой таблицы (ссылочная целостность);
- порядок, число и типы столбцов в ограничении должны соответствовать порядку, числу и типам целевых столбцов (целевые столбцы должны входить в ограничения PRIMARY KEY, UNIQUE или формировать уникальный индекс);
- таблица может содержать несколько ограничений внешнего ключа.

```
CREATE TABLE Sales.TempSalesReason (
  TempID INT NOT NULL,
  Name NVARCHAR(50),
  CONSTRAINT PK_TempSales
    PRIMARY KEY NONCLUSTERED (TempID),
  CONSTRAINT FK_TempSaltes_SalesReason FOREIGN KEY (TempID)
    REFERENCES Sales.SalesReason(SalesReasonID)
      ON DELETE CASCADE
      ON UPDATE CASCADE
);
```

Действия для ограничений внешнего ключа

- предложения REFERENCES инструкций CREATE TABLE и ALTER TABLE поддерживают предложения ON DELETE и ON UPDATE:
  - [ON DELETE { NO ACTION | CASCADE | SET NULL | SET DEFAULT }]
  - [ON UPDATE { NO ACTION | CASCADE | SET NULL | SET DEFAULT }]

- по умолчанию подразумевается действие NO ACTION (указывает, что при попытке удалить или изменить строку с ключом, на которую ссылаются внешние ключи в строках других таблиц, нужно сообщить об ошибке, а для соответствующей инструкции DELETE / UPDATE выполнить откат);
- действия CASCADE, SET NULL и SET DEFAULT позволяют удалять и изменять значения ключей в таблицах, ссылающихся на таблицу, в которую вносятся изменения:
  - CASCADE: каскадное изменение ссылающихся таблиц;
  - SET NULL: установка NULL для ссылающихся внешних ключей;
  - SET DEFAULT: установка значений по умолчанию для ссылающихся внешних ключей.

Ограничения множественных каскадных действий

- последовательности каскадных ссылочных действий, запускаемые отдельными инструкциями DELETE или UPDATE, должны образовывать без циклических ссылок;
- никакая таблица не должна появляться больше одного раза в списке всех каскадных ссылочных действий, вызванных инструкциями DELETE или UPDATE;
- кроме того, в дереве каскадных ссылочных действий к любой из задействованных таблиц должен быть только один путь;
- любая ветвь в дереве прерывается, как только встречается таблица, для которой указано действие NO ACTION или вообще не указано действие.

10. Представления: назначение, типы. Привязка к схеме. Обновление данных через представления.

Представления

- представление (view) - это виртуальная таблица, состоящая из совокупности именованных столбцов и строк данных, содержимое которой определяется запросом:
  - определяющий представление запрос может быть инициирован в одной или нескольких базовых таблицах или представлениях;
  - пока представление не будет материализовано / проиндексировано, оно не существует в базе данных как хранимая совокупность значений: строки и столбцы динамически формируются при обращении к представлению на основе данных из таблиц (и других представлений), указанных в определяющем представление запросе – т.е., фактически представление является сохранённым именованным запросом.

Функции представлений

- упрощение и настройка восприятия каждым пользователем информации базы данных;

- реализация механизмов безопасности, обеспечивающих возможность обращения пользователей к данным без предоставления им разрешений на непосредственный доступ к базовым таблицам;
- обеспечение интерфейса обратной совместимости, моделирующего таблицу, которая существует, но схема которой изменилась;
- определение представлений с данными из нескольких гетерогенных источников.

Типы представлений

- стандартные: сочетание данных из одной или нескольких таблиц;
- индексированные: материализованное представление (вычисленное и сохраненное в базе данных), изменения данных в базовых таблицах, отражаются в данных, хранимых в индексированном представлении;
- секционированные: объединение горизонтально секционированных данных набора базовых таблиц, находящихся на одном или нескольких серверах (локальное / распределенное секционированное представление).

Создание представлений

- определяющий представление запрос может быть инициирован в одной или нескольких базовых таблицах или представлениях;
- столбцу обязательно должно быть присвоено имя, если значение столбца представления формируется выражением и имя не было назначено в самом запросе;
- проверка схемы базовых объектов производится только при обращении к представлению.

```
CREATE VIEW hiredate_view
AS
SELECT p.FirstName, p.LastName, e.BusinessEntityID,
e.HireDate FROM HumanResources.Employee AS e
JOIN Person.Person AS p ON e.BusinessEntityID = p.BusinessEntityID;
CREATE VIEW Sales.SalesPersonPerform
AS
SELECT TOP (100) SalesPersonID, SUM(TotalDue) AS TotalSales FROM Sal
es.SalesOrderHeader
WHERE OrderDate > CONVERT(DATETIME,'20001231',101)
GROUP BY SalesPersonID;
```

Создание представлений: привязка к схеме

- при указании SCHEMABINDING базовые таблицы не могут быть изменены каким-либо образом, затрагивающим определение представления – при необходимости таких изменений представление должно быть создано повторно;
- все базовые объекты представления (таблицы, представления и функции) должны располагаться в одной базе данных;

- запрос, определяющий представление, должен использовать полные имена объектов (схема.объект).

```
CREATE OR ALTER VIEW Taxi.[Kamaz_cabs]
WITH SCHEMABINDING
AS
SELECT CabID, CarNumber
FROM Taxi.Cab
WHERE Brand = 'Kamaz'
```

Обновляемые представления

- любые изменения, в том числе инструкции UPDATE, INSERT и DELETE, должны ссылаться на столбцы только одной базовой таблицы;
- изменяемые в представлении столбцы должны прямо ссылаться на данные в столбце базовой таблицы - они не могут быть получены другим способом, например:
  - статистическими функциями (AVG, COUNT, SUM, MIN, MAX, GROUPING, STDEV, STDEVP, VAR или VARP);
  - вычислением на основе других столбцов (столбцы, полученные при помощи операторов UNION, UNION ALL, CROSSJOIN, EXCEPT и INTERSECT, также основаны на вычислениях и поэтому недоступны для изменения);
- изменяемые столбцы не могут указываться в предложениях GROUP BY, HAVING и DISTINCT.

Обновление представлений: CHECK OPTION

- при указании CHECK OPTION все изменения данных через представление должны удовлетворять ограничениям запроса, определяющего представление (таким образом, изменения данных не должны влиять на их видимость через представление).

```
CREATE VIEW dbo.SeattleOnly
AS
SELECT p.LastName, p.FirstName, e.JobTitle, a.City, sp.StateProvinceCod
e
FROM HumanResources.Employee e
INNER JOIN Person.Person p ON p.BusinessEntityID = e.BusinessEntityID
INNER JOIN Person.BusinessEntityAddress bea
ON bea.BusinessEntityID = e.BusinessEntityID INNER JOIN Person.Addres
s a
ON a.AddressID = bea.AddressID
INNER JOIN Person.StateProvince sp
ON sp.StateProvinceID = a.StateProvinceID
WHERE a.City = 'Seattle'
WITH CHECK OPTION;
```

11. Хранения данных: куча, кластеризованная таблица. Индексы на основе B-дерева. Некластеризованный индекс. Индексированное представление.

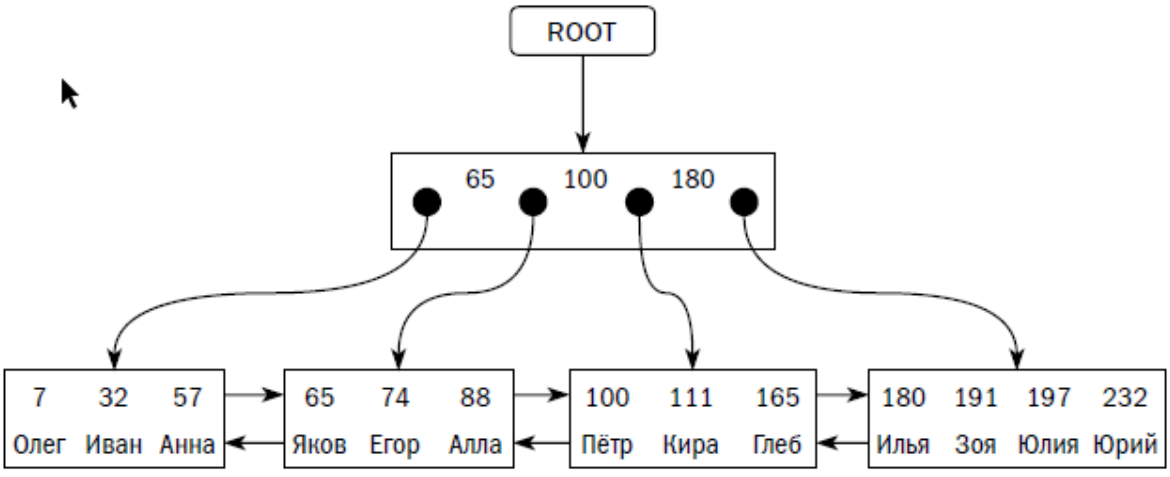
Индекс – структура на диске, которая связана с таблицей или представлением и может способствовать сокращению времени выполнения запроса за счет сокращения количества дисковых операций ввода-вывода и уменьшения потребления системных ресурсов;

- индекс содержит ключи, построенные на основе одного или нескольких столбцов в таблице или представлении;
- индексы могут быть уникальными (т.е. никакие две строки не имеют одинаковое значение для ключа индекса) или неуникальными;
- уникальные индексы создаются автоматически при определении ограничений PRIMARY KEY или UNIQUE на основе столбцов таблицы;
- основной индексной структурой, используемой для организации индексов, является B-дерево.

B-дерево

преимущества:

- эффективная вставка (количество изменяемых блоков не превышает глубины дерева);
- эффективный поиск (глубина дерева – log N / log f);
- поддерживает поиск по значению (равенство) и по диапазону (неравенства);
- результаты последовательного чтения индекса упорядочены по значению ключа индекса.



индексы на основе B-дерева эффективно поддерживают следующие варианты доступа:

- поиск уникального ключа;

- поиск множества ключей (неуникальный индекс, поиск по части ключа, поиск по диапазону ключей) – результат отсортирован по значению ключа;
- полный просмотр – результат отсортирован по значению ключа;

Также может быть эффективным быстрый просмотр индекса (просмотр всех страниц в случайном порядке как альтернатива полному сканированию таблицы).

Данные также могут храниться в виде

- неупорядоченной структуры (когда у таблицы нет кластеризованного индекса), которая называется кучей. Положение строки определяется комбинацией номера страницы и номера строки в странице.
- в кластеризованной таблице:
  - строки данных хранятся упорядоченно в кластеризованном индексе (индексе, листовые узлы которого хранят всю строку таблицы).
  - Строки данных хранятся упорядоченно в кластеризованном индексе.
  - Листовые узлы индекса хранят всю строку таблицы (представления), следовательно, строки отсортированы по значению ключа индекса
  - Может быть создан только один кластеризованный индекс для каждой таблицы (представления);
  - Положение строки определяется значением ключа.

Индексированные представления

- индексировать представление можно, создав для него уникальный кластеризованный индекс:

  – индексированные представления значительно повышают производительность некоторых типов запросов (например, группирующих множество строк);

  – индексированные представления не очень хорошо подходят для часто обновляющихся базовых наборов данных;

- запрос, формирующий индексированное представление:

  – должен ссылаться на единственную таблицу или соединение нескольких таблиц;

  – может выполнять агрегирование данных (GROUP BY) с ограничениями;

  – ключевые столбцы и условия WHERE и GROUP BY могут содержать только точные детерминированные выражения.

Индексированные представления: пример

```
SET NUMERIC_ROUNDABORT OFF;
SET ANSI_PADDING, ANSI_WARNINGS, CONCAT_NULL_YIELDS_NULL, ARITHABORT,
QUOTED_IDENTIFIER, ANSI_NULLS ON;
CREATE VIEW Sales.vOrders WITH SCHEMABINDING
AS
SELECT SUM(UnitPrice * OrderQty * (1.00 - UnitPriceDiscount)) AS Revenue,
```

```
OrderDate, ProductID, COUNT_BIG(*) AS COUNT
FROM Sales.SalesOrderDetail AS od, Sales.SalesOrderHeader AS o
WHERE od.SalesOrderID = o.SalesOrderID
GROUP BY OrderDate, ProductID;
GO
CREATE UNIQUE CLUSTERED INDEX IDX_V1 ON Sales.vOrders ( OrderDate, ProductID
);
GO
SELECT OrderDate, SUM(UnitPrice * OrderQty * (1.00 - UnitPriceDiscount)) AS
Rev FROM Sales.SalesOrderDetail AS od INNER JOIN Sales.SalesOrderHeader AS o
ON od.SalesOrderID = o.SalesOrderID
AND o.OrderDate >= CONVERT(DATETIME, '03/01/2012', 101)
AND o.OrderDate < CONVERT(DATETIME, '04/01/2012', 101)
GROUP BY OrderDate
ORDER BY OrderDate ASC;
```

Некластеризованные индексы

- некластеризованные (**non-clustered**) индексы имеют структуру, отдельную от строк данных;
- в некластеризованном индексе содержатся значения его ключа, и каждая запись ключа в листовой странице содержит указатель строки (row locator) на соответствующую строку данных:
  - для кластеризованной таблицы указатель строки является ключом кластеризованного индексе;
  - для кучи указатель строки является идентификатором строки (RID, **row identifier**);
- добавление дополнительных неключевых столбцов на конечный уровень некластеризованного индекса позволяет выполнять полностью индексированные (покрывающие, covering) запросы.

12. Проектирование и использование индексов. Уникальные, покрывающие и частичные индексы.

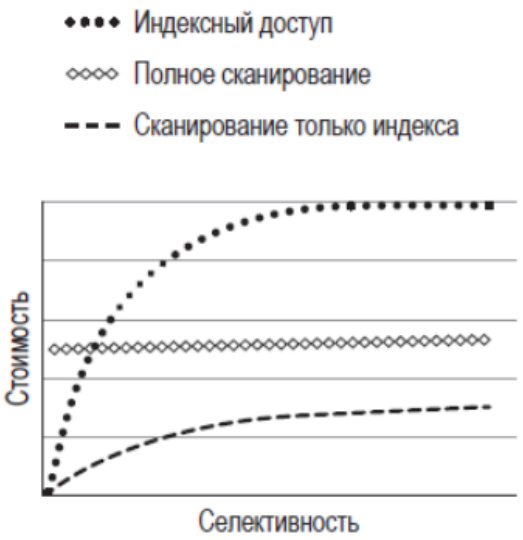
Предпосылки использования индексов

- индексы в СУБД предназначены для:
  - ускорения поиска и получения данных по определённым критериям.
  - поддержания уникальности значений

Использование индексов

- необходимость использования индексов, их набор и характеристики определяются исходя из набора запросов;
- преимущества: время отклика;
- Недостатки:
  - накладные расходы на хранение;
  - накладные расходы на обновление;
  - накладные расходы в процессе

- индексы используются, если:
  - низкая селективность индекса (высокая кардинальность значений в таблице);
  - покрывающий / составной индекс;
- индексы не используются, если:
  - для выполнения запроса нужна большая часть строк таблицы
  - небольшая таблица полностью считывается в оперативную память.



Проектирование индексов

- при проектировании и создании индексов учитываются следующие аспекты:
  - набор ключевых полей индекса;
  - порядок сортировки индекса;
  - уникальность индекса;
  - добавление неключевых столбцов к конечному уровню некластеризованного индекса;
  - количество, размер и типы столбцов в ключе индекса;
  - размещение индексов в файловых группах.

Уникальные индексы

- уникальный кластеризованный индекс автоматически создаётся при определении ограничения первичного ключа (PRIMARY KEY);
- уникальный некластеризованный индекс автоматически создаётся при определении ограничения UNIQUE (значение NULL также считается уникальным и может встречаться не более одного раза).

```
CREATE CLUSTERED INDEX IX_TestT_TestC ON dbo.TestTable (TestCol1);
CREATE NONCLUSTERED INDEX IX_ProductVendor_VendorID ON
Purchasing.ProductVendor (BusinessEntityID);
IF EXISTS (SELECT name from sys.indexes
WHERE name = N'AK_UnitMeasure_Name'
AND object_id = OBJECT_ID(N'Production.UnitMeasure', N'U')) DROP INDEX
AK_UnitMeasure_Name ON Production.UnitMeasure;
```

```
GO
CREATE UNIQUE INDEX AK_UnitMeasure_Name ON Production.UnitMeasure (Name);
```

Покрывающие индексы

- покрывающие (**covering**) индексы могут существенно повысить производительность запросов за счёт того, что все столбцы, необходимые для выполнения запроса, присутствуют в индексе (следовательно, не требуются обращения к основной таблице)
  - составные индексы;
  - индексы с дополнительными неключевыми полями.

```
CREATE INDEX IX_VendorID
ON dbo.ProductVendor (VendorID DESC, Name ASC, Address DESC);
CREATE NONCLUSTERED INDEX IX_Address_PostalCode
ON Person.Address (PostalCode)
INCLUDE (AddressLine1, AddressLine2, City, StateProvinceID); GO
SELECT AddressLine1, AddressLine2, City, StateProvinceID, PostalCode FROM
Person.Address
WHERE PostalCode BETWEEN N'98000' and N'99999';
GO
```

Частичные индексы

- частичные (**filtered**) индексы позволяют проиндексировать только некоторое подмножество строк таблицы (представляющих наибольший интерес с точки зрения бизнес-логики приложения, использующего данные).

```
DROP INDEX IF EXISTS FIBillOfMaterialsWithEndDate ON
Production.BillOfMaterials
GO
CREATE NONCLUSTERED INDEX FIBillOfMaterialsWithEndDate ON
Production.BillOfMaterials (ComponentID, StartDate) WHERE EndDate IS NOT
NULL ;
GO
SELECT ProductAssemblyID, ComponentID, StartDate FROM
Production.BillOfMaterials
WHERE EndDate IS NOT NULL
AND ComponentID = 5
AND StartDate > '01/01/2008';
```

13. Основные элементы синтаксиса Transact-SQL: управление выполнением, обработка ошибок.

Управление выполнением

- Язык Transact-SQL содержит специальные ключевые слова, известные как язык управления выполнением, которые управляют порядком выполнения инструкций на языке Transact-SQL, блоками инструкций и хранимыми процедурами;
- инструкции управления выполнением не могут быть распределены по разным пакетам или хранимым процедурам; существуют следующие ключевые слова, управляющие выполнением:

- BEGIN…END
- BREAK
- GOTO
- CONTINUE
- IF…ELSE
- WHILE
- RETURN
- WAITFOR

Составной оператор (BEGIN...END)

- инструкции BEGIN и END используются для группирования нескольких инструкций Transact-SQL в одном логическом блоке;
- инструкции BEGIN и END используются, когда:
  - в цикл WHILE необходимо включить блок инструкций;
  - в элемент функции CASE необходимо включить блок инструкций;
  - в предложение IF или ELSE необходимо включить блок инструкций

```
IF (@@ERROR <> 0)
BEGIN
SET @ErrorSaveVariable = @@ERROR
PRINT 'Error encountered, ' +
CAST(@ErrorSaveVariable AS VARCHAR(10))
END
```

Ветвления и переходы

- ветвление (проверка выполнения условия):

```
IF expression
  statement
ELSE
  statement
```

- безусловное завершение запроса, хранимой процедуры или пакета: RETURN [**@ReturnCode**]
- безусловный переход к метке:

```
GOTO label ...
label_name: ...
```

Организация цикла

- цикл:

```
WHILE expression
  statement
```

- управление циклом:
  - завершение цикла BREAK

- переход к очередной итерации CONTINUE

Инструкция WAITFOR

- инструкция WAITFOR приостанавливает выполнение пакета, хранимой процедуры или транзакции до тех пор, пока:
  - не пройдет заданный интервал времени: DELAY(**time_to_pass**)
  - не наступит заданное время: TIME(**time_to_execute**)
  - инструкция RECEIVE не изменит или не возвратит, по крайней мере одну строку в очередь компонента Service Broker: RECEIVE / TIMEOUT

Обработка ошибок

- ошибки, вызываемые компонентом Database Engine, можно обрабатывать на двух уровнях:
  - в компоненте Database Engine, вводя код обработки ошибок в пакеты Transact-SQL, хранимые процедуры, триггеры или в пользовательские функции. Механизмы обработки ошибок Transact-SQL включают:
    - конструкцию TRY…CATCH…END CATCH (получение информации об ошибке с помощью функций ERROR_LINE, ERROR_MESSAGE, ERROR_NUMBER, ERROR_PROCEDURE, ERROR_SEVERITY и ERROR_STATE)
    - инструкции RAISERROR / THROW / PRINT (возврат приложению сообщения в формате системных ошибок или предупреждений / в формате символьной строки)
    - функцию @@ERROR (получение номера ошибки, созданной предыдущей инструкцией языка Transact-SQL)
  - на уровне вызывающего приложения: каждый из API, используемых приложением для доступа к компоненту Database Engine, наделен механизмами для возвращения сведений об ошибках соответствующему приложению.

14. Хранимые процедуры: типы, обработка.

Хранимые процедуры

- хранимые процедуры в Microsoft SQL Server аналогичны процедурам в других языках программирования:
  - обрабатывают входные аргументы и возвращают вызывающей процедуре или пакету значения в виде выходных аргументов (в том числе через параметры типа cursor);
  - содержат программные инструкции, которые выполняют операции в базе данных, в том числе вызывающие другие процедуры;

  - возвращают значение состояния (код результата) вызывающей процедуре или пакету, таким образом передавая сведения об успешном или неуспешном завершении;
- по сравнению с программами Transact-SQL, которые хранятся локально на клиентских компьютерах, хранимые процедуры SQL Server имеют следующие преимущества:
  - регистрируются на сервере;
  - могут иметь атрибуты безопасности;
  - позволяют сделать защиту приложений более надежной;
  - поддерживают модульное программирование;
  - представляют собой именованный код, дающий возможность отсроченного связывания;
  - позволяют уменьшить сетевой трафик.

Типы хранимых процедур

- пользовательские хранимые процедуры:
  - Transact-SQL: сохраненная коллекция инструкций языка Transact-SQL, которая может принимать и возвращать параметры;
  - CLR: ссылка на метод общеязыковой среды выполнения (CRL) платформы Microsoft .NET Framework, который может принимать и возвращать пользователю параметры;
- расширенные хранимые процедуры (устарели и не будут поддерживаться в следующих выпусках): библиотеки DLL, которые могут динамически загружаться и выполняться экземпляром Microsoft SQL Server;
- системные хранимые процедуры: специальные процедуры, позволяющие осуществлять административные действия в SQL Server:
  - физически системные хранимые процедуры хранятся в базе данных ресурсов и имеют префикс sp_...;
  - логически же они отображаются в любой системной или пользовательской базе данных в схеме sys;
  - SQL Server поддерживает расширенные системные хранимые процедуры (имеют префикс xp_), обеспечивающие интерфейс между SQL Server и внешними программами для выполнения различных действий по обслуживанию системы.

Создание и удаление хранимых процедур

```
CREATE PROCEDURE procedure_name [ ; number ] [ { @parameter data_type }
[ VARYING ] [ = default ] [ OUTPUT ] ] [ ,...n ]
AS
sql_statement [ ...n ]
IF OBJECT_ID ('HumanResources.usp_GetAllEmployees', 'P' ) IS NOT NULL DROP
PROCEDURE HumanResources.usp_GetAllEmployees;
GO
CREATE PROCEDURE HumanResources.usp_GetAllEmployees AS SELECT LastName,
FirstName, JobTitle, Department FROM HumanResources.vEmployeeDepartment;
GO
```

```
EXECUTE HumanResources.usp_GetAllEmployees;
GO
-- Or
EXEC HumanResources.usp_GetAllEmployees;
GO
-- Or, if this procedure is the first statement within a batch:
HumanResources.usp_GetAllEmployees;
```

Хранимые процедуры с подстановочными параметрами

```
IF OBJECT_ID ( 'HumanResources.usp_GetEmployees2', 'P') IS NOT NULL DROP
PROCEDURE HumanResources.usp_GetEmployees2;
GO
CREATE PROCEDURE HumanResources.usp_GetEmployees2
@lastname varchar(40) = 'D%',
@firstname varchar(20) = '%'
AS
SELECT LastName, FirstName, JobTitle, Department FROM
HumanResources.vEmployeeDepartment
WHERE FirstName
LIKE @firstname AND LastName LIKE @lastname; GO
EXECUTE HumanResources.usp_GetEmployees2 'Wi%';
-- Or
EXECUTE HumanResources.usp_GetEmployees2 @firstname = '%'; -- Or
EXECUTE HumanResources.usp_GetEmployees2 '[CK]ars[OE]n'; -- Or
EXECUTE HumanResources.usp_GetEmployees2 'Hesse', 'Stefen'
```

Возвращение данных с использованием параметров OUTPUT и кода возврата

- если при выполнении хранимой процедуры указано OUTPUT для параметра, а параметр не указан при помощи OUTPUT в хранимой процедуре, выдается сообщение об ошибке;
- можно выполнять хранимую процедуру с параметрами OUTPUT и не указывать OUTPUT при выполнении хранимой процедуры: сообщение об ошибке не будет выдаваться, но нельзя будет использовать выходное значение в вызывающей программе.

```
CREATE PROCEDURE Sales.usp_GetEmployeeSalesYTD
@SalesPerson nvarchar(50),
@SalesYTD money OUTPUT
AS
SELECT @SalesYTD = SalesYTD
FROM Sales.SalesPerson AS sp
JOIN HumanResources.vEmployee AS e ON e.EmployeeID = sp.SalesPersonID WHERE
LastName = @SalesPerson;
RETURN
GO
DECLARE @SalesYTDBySalesPerson money;
EXECUTE @result = Sales.usp_GetEmployeeSalesYTD
N'Blythe', @SalesYTD = @SalesYTDBySalesPerson OUTPUT; PRINT 'YTD sales' +
convert(varchar(10),@SalesYTDBySalesPerson); GO
```

15. Курсоры: назначение, типы, обработка.

Курсоры являются расширением результирующих наборов, которые предоставляют механизм, позволяющий обрабатывать одну строку или небольшое их число за один раз; курсоры обеспечивают возможность:

- позиционирования на отдельные строки результирующего набора;
- получения одной или нескольких строк от текущей позиции в результирующем наборе;
- изменения данных в строках в текущей позиции результирующего набора.

Типы курсоров

- статические (static):
  - результирующий набор фиксируется при открытии курсора (не отражает изменения, влияющие на вхождение в результирующий набор, изменения значений, а также изменения в результате удаления строк в базе данных);
  - в Microsoft SQL Server доступны только для чтения;
- основанные на потенциальном ключе (keyset):
  - набор записей фиксируется при открытии: членство и порядок строк являются фиксированными при открытии курсора;
  - курсоры управляются с помощью набора уникальных идентификаторов – ключей, которые создаются из набора столбцов, уникально идентифицирующего строки результирующего набора;
  - изменения в значениях столбцов, не входящих в ключевой набор (внесенные владельцем курсора или другими пользователями), становятся видны при прокрутке курсора;
  - вставка в базу данных, выполненная вне курсора, видна только после его повторного открытия;
  - при попытке запросить состояние строки, удаленной после открытия курсора, функция @@FETCH_STATUS возвращает состояние “строка отсутствует“;
  - обновление ключевого столбца действует аналогично удалению старого значения ключа с последующей вставкой нового значения (если обновление не было выполнено с помощью курсора, то новое ключевое значение не будет видимо);
- динамические:
  - набор записей не фиксируется при открытии курсора (отражаются все изменения строк в результирующем наборе при прокрутке курсора: значения типа данных, порядок и членство строк в результирующем наборе могут меняться для каждой выборки).

Обработка курсоров

- курсоры Transact-SQL и API-курсоры имеют различный синтаксис, но для всех курсоров SQL Server используется одинаковый цикл обработки:

1. связать курсор с результирующим набором инструкции Transact-SQL и задать его характеристики (например, возможность обновления строк);

2. выполнить инструкцию Transact-SQL для заполнения курсора;
3. получить в курсор необходимые строки:
   1. операция получения в курсор одной и более строк называется выборкой;
   2. выполнение серии выборок для получения строк в прямом или обратном направлении называется прокруткой;
4. при необходимости выполнить операции изменения (обновления или

удаления) строки в текущей позиции курсора;
5. закрыть курсор.

- однонаправленный курсор не поддерживает прокрутку, а только последовательную выборку строк от начала курсора до его конца:
- строки не извлекаются из базы данных, пока они не будут выбраны;
- результаты всех инструкций INSERT, UPDATE и DELETE, которые влияют на строки результирующего набора и выполненные текущим пользователем или зафиксированы другими пользователями, отображаются в процессе выборки строк из курсора.

16. Определяемые пользователем функции: скалярные функции. Свойства функций и их влияние на эффективность использования функций.

Определяемые пользователем функции

- преимущества использования:
  - возможность модульного программирования;
  - ускорение выполнения за счет кэширования и повторного использования планов выполнения;
  - уменьшение сетевого трафика;
- ограничение использования функций: инструкции внутри функции могут изменять только локальные по отношению к этой функции объекты, например, локальные курсоры или переменные.

Компоненты пользовательской функции

- функция принимает ноль и более параметров и возвращает:
  - скалярное значение;
  - таблицу;
- любая пользовательская функция состоит из двух частей:
  - заголовка, содержащего:
  - имя функции с необязательным именем схемы или владельца;
  - имя и тип данных входного параметра;
  - тип данных возвращаемого значения и необязательное имя;
  - тела, состоящего из:
  - одной или нескольких инструкций Transact-SQL, реализующих логику функции;
  - ссылки на сборку .NET .

Определяемые пользователем функции: пример

```
CREATE FUNCTION dbo.GetWeekDay (@Date datetime) RETURNS int
AS
BEGIN
RETURN DATEPART (weekday, @Date)
END;
GO
SELECT dbo.GetWeekDay(CONVERT(DATETIME,'20020201',101)) AS DayOfWeek;
```

Виды определяемых пользователем функций

- скалярные функции:
  - возвращают единственное значение скалярного типа;
  - могут использоваться везде, где допустимы выражения возвращаемого типа;
- табличные функции:
  - возвращают значение типа table;
  - могут использоваться везде, где допустимы табличные выражения (в том числе, как альтернатива представлений и хранимых процедур, возвращающих один результирующий набор).

Свойства функций

- свойства пользовательских функций определяют способность ядра СУБД индексировать результаты функции как с помощью индексов вычисляемых столбцов, вызывающих функцию, так и с помощью индексированных представлений, которые на нее ссылаются:
  - детерминизм: способность возвращать один и тот же результат, если предоставлять им один и тот же набор входных значений и использовать одно и то же состояние базы данных;
  - точность: отсутствие операций над числами с плавающей запятой;
  - доступ к данным: осуществление доступа к локальному серверу баз данных с помощью внутрипроцессного управляемого поставщика SQL Server;
  - доступ к системным данным: осуществление доступа к системным метаданным на локальном сервере баз данных с помощью внутрипроцессного управляемого поставщика SQL Server
  - свойство IsSystemVerified: определяет, может ли ядро СУБД проверить детерминизм и точность функции.

Скалярные функции: синтаксис и пример

```
CREATE FUNCTION function_name
( [ { @parameter_name [AS]
scalar_parameter_data_type } [ ,...n ] ] ) RETURNS scalar_return_data_type
[ AS ]
BEGIN
function_body
RETURN scalar_expression
END
```

```
CREATE FUNCTION dbo.ufnGetStock(@ProductID int)
RETURNS int
AS
BEGIN
DECLARE @ret int;
SELECT @ret = SUM(p.Quantity)
FROM Production.ProductInventory p
WHERE p.ProductID = @ProductID AND p.LocationID = '6'; IF (@ret IS NULL)
SET @ret = 0
RETURN @ret
END;
GO
```

17. Определяемые пользователем функции: табличные функции. Сравнение функций и хранимых процедур.

Табличные функции: возвращают значение типа table; могут использоваться везде, где допустимы табличные выражения (в том числе, как альтернатива представлений и хранимых процедур, возвращающих один результирующий набор). А сравнений нигде нет, всем удачи

```
CREATE FUNCTION function_name
( [ { @parameter_name [AS]
scalar_parameter_data_type } [ ,...n ] ] )
RETURNS @return_variable TABLE < table_type_definition >
[ AS ]
BEGIN
function_body
RETURN
END
---- inline
CREATE FUNCTION function_name
( [ { @parameter_name [AS]
scalar_parameter_data_type } [ ,...n ] ] )
RETURNS TABLE
[ AS ]
RETURN [ ( ] select-statement [ ) ]
```

В лекциях не было, ноооо

Критерий	Пользовательская Функция (UDF)	Хранимая Процедура (SP)
Назначение	**Вычисление и возврат значения.** Аналогична формуле.	**Выполнение действий.** Аналогична программе или скрипту.
Возвращаемое значение	**Обязательно** возвращает одно скалярное значение **или** таблицу.	**Не обязана** возвращать значение. Может возвращать: • 0 или N результирующих наборов (SELECT). • Целочисленный код возврата (RETURN). • Выходные параметры (OUTPUT).
Использование в SQL	Может использоваться **в любом месте, где допустимо выражение**: • В SELECT (в списке столбцов, WHERE, HAVING). • В JOIN (табличные функции). • В CHECK/DEFAULT-ограничениях. • В определении вычисляемого столбца.	Вызывается **только как отдельная инструкция** с `EXEC/EXECUTE` или из другого кода. Нельзя встроить в SELECT.
Изменение данных	**Запрещено.** Функция не может модифицировать состояние базы данных (INSERT, UPDATE, DELETE, DDL). Может работать только с **временными таблицами** (локальными).	**Разрешено.** Может выполнять любые операции с данными и объектами.
Обработка ошибок	Ограничена. Нельзя использовать `TRY...CATCH` внутри функции.	Полноценная. Поддерживается `TRY...CATCH` для сложной обработки ошибок.
Транзакции	Не может управлять транзакциями (BEGIN/COMMIT/ROLLBACK TRAN).	Может полностью управлять транзакциями.
Параметры	Могут быть только **входными** (IN). Не поддерживает OUTPUT-параметры.	Поддерживает **входные (IN), выходные (OUTPUT)** и **входо-выходные** параметры.
Производительность	Может быть **инлайнирована** (встроена в запрос оптимизатором), особенно встроенные табличные функции (inline TVF). Многократный вызов в одном запросе может привести к потере производительности.	Выполняется как отдельный модуль. Может использовать **кеширование плана выполнения**, что эффективно для сложных операций.
Свойства	Имеют важные свойства: **детерминированность**, **точность**, которые влияют на возможность индексирования (в вычисляемых столбцах и индексированных представлениях).	Не имеют подобных формальных свойств.

18. Триггеры: назначение, типы. Триггеры DDL.

Триггером называют хранимую процедуру особого типа, которая автоматически выполняется при возникновении языкового события;

- триггеры являются одним из двух механизмов реализации бизнес-правил и целостности данных (наряду с ограничениями);
- типы триггеров:
  - DML-триггер: действие, которое выполняется при наступлении события языка DML (инструкции UPDATE, INSERT и DELETE, выполняемые в таблице или представлении) на сервере базы данных - используются для расширения бизнес-правил при изменении данных, а также для расширения логики проверки целостности ограничений, правил и значений по умолчанию;
  - DDL-триггер: действие, которое выполняется при возникновении событий языка определения данных (DDL) на сервере баз данных;
- триггер и инструкция, при выполнении которой он срабатывает, считаются одной транзакцией, которую можно откатить назад внутри триггера;

Триггеры DML: варианты использования

- для каскадных изменений в связанных таблицах базы данных; (данные изменения можно более эффективно выполнить при помощи каскадных ограничений ссылочной целостности);
- для предотвращения случайных или некорректных операций INSERT, UPDATE и DELETE;

- для реализации других более сложных ограничений, чем определенных при помощи ограничения CHECK (в отличие от ограничений CHECK, DML-триггеры могут ссылаться на столбцы других таблиц);
- для оценки состояния и анализа таблицы до и после изменения данных и выполнения действия на основе этого различия;
- несколько DML-триггеров одинакового типа (INSERT, UPDATE или DELETE) для таблицы позволяют предпринять несколько различных действий в ответ на одну инструкцию изменения данных;
- для реализации более сложных методов управления ошибками (по сравнению со стандартными системными сообщениями);
- для определения необходимых действий по обновлению представления (расширение типов обновлений, которые поддерживает представление).

Триггеры DDL

- триггеры DDL - особый вид триггеров, которые срабатывают при выполнении инструкций языка DDL (CREATE, ALTER, DROP и т.п.)
- триггеры DDL могут применяться при выполнении административных задач, например, для аудита и регулирования операций в базе данных:
  - для предотвращения внесений определенных изменений в схему базы данных;
  - для выполнения в базе данных некоторых действий в ответ на изменения в схеме базы данных;
  - для записи изменений или событий схемы базы данных;
- триггеры DDL срабатывают только после выполнения соответствующих инструкций DDL;
- триггер DDL и инструкция, приводящая к его срабатыванию, выполняются в одной транзакции;
- для одной инструкции Transact-SQL можно создать несколько триггеров DDL.

Триггеры DDL: пример

```
CREATE TRIGGER safety
ON DATABASE
FOR DROP_TABLE, ALTER_TABLE
AS
PRINT 'You must disable Trigger "safety" to drop or alter tables!' ROLLBACK ;

CREATE TRIGGER ddl_trig_login
ON ALL SERVER
FOR DDL_LOGIN_EVENTS
AS
PRINT 'Login Event Issued.'
SELECT EVENTDATA().value('(/EVENT_INSTANCE/TSQLCommand/CommandText)[1]',
'nvarchar(max)')
GO
DROP TRIGGER ddl_trig_login
ON ALL SERVER
GO;
```

19. Триггеры DML: триггеры типа INSTEAD OF.

- триггеры INSTEAD OF:
  - используются в таблицах и представлениях (с одной или более базовыми таблицами);
  - выполняются вместо обычных действий, запускающих триггеры, при повторном вызове действия триггер не срабатывает;
  - выполняются до проверки ограничений;
  - единственный триггер на каждое из запускающих триггеры действие (UPDATE, DELETE или INSERT);
  - не допускаются в таблицах, в которых используется каскадное обновление (для UPDATE и DELETE);
  - могут работать со столбцами типа text, ntext и image;

Создание триггера DML

```
CREATE TRIGGER trigger_name
ON { table | view }
{ FOR | AFTER | INSTEAD OF }
{ [ DELETE ] [ , ] [ INSERT ] [ , ] [ UPDATE ] } AS
sql_statement [ ...n ]
CREATE TRIGGER TableAInsertTrig ON TableA INSTEAD OF INSERT AS ...
CREATE TRIGGER TableBDeleteTrig ON TableB AFTER DELETE AS ...

uses the FOR keyword to generate an AFTER trigger.

CREATE TRIGGER TableCUpdateTrig ON TableC FOR UPDATE AS ...
```

Использование триггеров DML: пример

```
CREATE TABLE my_table (a int NULL, b int NULL) GO
CREATE TRIGGER my_trig
ON my_table
FOR INSERT
AS
IF UPDATE(b)
PRINT 'Column b Modified'
GO
CREATE TRIGGER my_trig2
ON my_table
FOR INSERT
AS
IF ( COLUMNS_UPDATED() & 2 = 2 )
PRINT 'Column b Modified'
GO
```

Триггеры DML: работа с одной и несколькими строками

```
CREATE TRIGGER NewPODetail
ON Purchasing.PurchaseOrderDetail
AFTER INSERT
AS
UPDATE PurchaseOrderHeader
```

```
SET SubTotal = SubTotal + LineTotal
FROM inserted
WHERE PurchaseOrderHeader.PurchaseOrderID = inserted.PurchaseOrderID;
CREATE TRIGGER NewPODetail2
ON Purchasing.PurchaseOrderDetail
AFTER INSERT
AS
UPDATE PurchaseOrderHeader SET SubTotal = SubTotal +
(SELECT SUM(LineTotal) FROM inserted WHERE
PurchaseOrderHeader.PurchaseOrderID = inserted.PurchaseOrderID) WHERE
PurchaseOrderHeader.PurchaseOrderID
IN (SELECT PurchaseOrderID FROM inserted);
```

20. Триггеры DML: триггеры типа AFTER.

- триггеры AFTER (или FOR):
  - могут применяться только в таблицах;
  - выполняются после выполнения инструкций INSERT, UPDATE или DELETE;
  - выполняются после проверки ограничений;
  - может быть более одного для каждого действия (UPDATE, DELETE или INSERT), можно задать выполнение в первую и в последнюю очередь;
  - не могут работать со столбцами типа text, ntext и image;

21. Триггеры DML: сравнение триггеров тип INSTEAD OF и AFTER. Дополнительные средства управления данными, доступные в триггерах.

- триггеры INSTEAD OF:
  - используются в таблицах и представлениях (с одной или более базовыми таблицами);
  - выполняются вместо обычных действий, запускающих триггеры, при повторном вызове действия триггер не срабатывает;
  - выполняются до проверки ограничений;
  - единственный триггер на каждое из запускающих триггеры действие (UPDATE, DELETE или INSERT);
  - не допускаются в таблицах, в которых используется каскадное обновление (для UPDATE и DELETE);
  - могут работать со столбцами типа text, ntext и image.

- триггеры AFTER (или FOR):
  - могут применяться только в таблицах;
  - выполняются после выполнения инструкций INSERT, UPDATE или DELETE;
  - выполняются после проверки ограничений;
  - может быть более одного для каждого действия (UPDATE, DELETE или INSERT), можно задать выполнение в первую и в последнюю очередь;
  - не могут работать со столбцами типа text, ntext и image;

Специальные средства, доступные в триггерах

- таблицы inserted и deleted (соответственно, вставленных и удаленных значений), формат таблиц соответствует формату таблицы или представления, для которых задан триггер;
- функция COLUMNS_UPDATED() возвращает битовую маску обновленных столбцов;
- функция UPDATE(column_name) возвращает TRUE, если столбец был обновлен.

Вопрос 2

1. Управление параллельной обработкой в БД. Проблемы (аномалии) параллельной обработки транзакций.

Управление параллельной обработкой (управление параллелизмом, concurrency control) направлено на то, чтобы исключить непредусмотренное влияние действий одного пользователя на действия другого.

Управление параллельной обработкой, как правило, предполагает компромисс между уровнем изоляции различных операций друг от друга и производительностью системы.

Параллельные транзакции (parallel transactions) – две или более транзакций, обрабатываемые одновременно.

- **Проблемы параллельной обработки**:
  - грязное / несогласованное чтение (dirty / inconsistent reads);
  - невоспроизводимое / фантомное чтение (unrepeatable / phantom reads);
  - потерянное обновление (lost / concurrent update);

Необходимо упорядочить операции внутри транзакций таким образом, чтобы предотвратить нежелательное влияние одной транзакции на другую.

2. Свойства транзакций. Сериализуемость транзакций.

Транзакция обладает четырьмя свойствами (ACID):

- атомарность (atomicity): транзакция должна быть атомарной единицей работы; должны быть выполнены либо все входящие в нее модификации данных, либо ни одно из них не должно быть выполнено;
- согласованность (consistency): по завершении транзакция должна оставить все данные в согласованном состоянии (включая все внутренние структуры данных), для этого модификации должны применяться к данным в том состоянии, в котором они были на момент начала операции:
  - на уровне оператора (statement-level consistency);
  - на уровне транзакции (transaction-level consistency);
- изоляция (isolation): выполняемые транзакцией модификации должны быть (могут быть) изолированы от любых модификаций, проводимых другими транзакциями;

- устойчивость (durability): после завершения транзакции произведенные ею действия фиксируются в системе (даже в случае системного сбоя).

Сериализуемость транзакций (serializable transactions) – такая схема обработки транзакций, при которой результат их параллельной обработки такой же, как если бы они выполнялись последовательно (то есть, существует эквивалентная относительно конфликтов последовательность операций, соответствующая последовательному выполнению транзакций).

3. Стандарт SQL-92: аномалии изоляции транзакций и уровни изоляции.

Стандарт SQL-92: аномалии изоляции транзакций

- проблемы (аномалии, **phenomena**) изоляции транзакций, рассматриваемые стандартом SQL-92:
  - **«грязное» чтение** (**dirty read**) – чтение транзакцией записи, измененной другой транзакцией, при этом эти изменения еще не зафиксированы;
  - **невоспроизводимое чтение** (**non-repeatable read**) – при повторном чтении транзакция обнаруживает измененные или удаленные данные, зафиксированные другой транзакцией;
  - **фантомное чтение** (**phantom read**) – при повторном чтении транзакция обнаруживает новые строки, вставленные другой завершенной транзакцией;

Стандарт SQL-92: уровни изоляции транзакций

- стандарт SQL-92 определяет четыре уровня изоляции транзакций:
  - незавершенное чтение (**read uncommitted**);
  - завершенное чтение (**read committed**);
  - воспроизводимое чтение (**repeatable read**);
  - сериализуемость (**serializable** [**anomaly-serializable**]);

		Isolation Level			
		Read Uncommitted	Read Committed	Repeatable Read	Serializable
Problem Type	Dirty Read	Possible	Not Possible	Not Possible	Not Possible
	Nonrepeatable Read	Possible	Possible	Not Possible	Not Possible
	Phantom Read	Possible	Possible	Possible	Not Possible

Изоляция моментальных снимков

- изоляция моментальных снимков (snapshot isolation) позволяет транзакциям получать согласованный набор данных, существовавший на момент начала транзакции – версионирование строк (**row versioning**);
- таким образом, чтение данных никогда не блокируется записью, но данные могут быть неактуальными;

- фиксация транзакции происходит только если изменяемые данные не были изменены (зафиксированы) другой транзакцией.

4. Стратегии и механизмы управления параллелизмом. Блокировка ресурсов. Алгоритм Двухфазной блокировки. Взаимная блокировка.

Механизмы синхронизации:

- блокировка (resource-locking): взаимоисключающий доступ к разделяемым данным;
- упорядочивание транзакций в соответствии с набором правил (transaction ordering), в том числе упорядочивание по временным меткам (TO, timestamp ordering);

Стратегии, обеспечивающие изоляцию транзакций:

- пессимистическая (pessimistic concurrency control): обнаружение и предотвращение конфликтов в процессе выполнения транзакции;
- оптимистическая блокировка (optimistic concurrency control): анализ конфликтов в момент фиксации транзакции.

Блокировка ресурсов

- инициатор блокировки:
  - СУБД: неявная блокировка (implicit lock);
  - программа / запрос: явная блокировка (explicit lock);
- степень (глубина) детализации блокировки (locking granularity):
  - на уровне базы данных;
  - на уровне таблицы;
  - на уровне страницы;
  - на уровне строки;
- режим блокировки (lock mode):
  - монопольный (exclusive lock / write lock): блокируются все виды доступа к элементу;
  - разделяемый (shared lock / read lock): блокируется только запись.

Один из способов достижения сериализуемости – использование **двухфазной блокировки** (two-phase locking):

- фаза нарастания (growing phase): транзакция может налагать блокировки по мере необходимости, до тех пор, пока не снимается хотя бы одна блокировка;
- фаза сжатия (shrinking phase): после снятия одной из блокировок никакие другие накладываться уже не могут;

Модификация двухфазной блокировки (S2PL – strict 2PL): снятие блокировок непосредственно в момент фиксации / отката транзакции (предотвращает каскадные откаты).

Взаимная блокировка (deadlock, deadly embrace) – состояние взаимного ожидания транзакциями освобождения ресурсов, заблокированных друг другом; анализ: граф ожидания (WFG, wait-for graph); Обработка взаимной блокировки:

- предотвращение (deadlock prevention): блокирование всех ресурсов одновременно в начале операции;
- избежание (deadlock avoidance): требование блокировки в одинаковом порядке;
- обнаружение и устранение (deadlock detection and resolution): откат одной или нескольких транзакций, вызвавших блокировку (victim transaction) – оценка стоимости отката / стоимости завершения / количества разрешаемых блокировок (циклов).

5. Стратегии и механизмы управления параллелизмом. Многоверсионное управление параллельным доступом.

Стратегии и механизмы управления параллелизмом

- механизмы синхронизации:
  - блокировка (**resource-locking**): взаимоисключающий доступ к разделяемым данным;
  - упорядочивание транзакций в соответствии с набором правил (**transaction ordering**), в том числе упорядочивание по временным меткам (**TO, timestamp ordering**);
- стратегии, обеспечивающие изоляцию транзакций:
  - пессимистическая (**pessimistic concurrency control**): обнаружение и предотвращение конфликтов в процессе выполнения транзакции;
  - оптимистическая блокировка (**optimistic concurrency control**): анализ конфликтов в момент фиксации транзакции.

Многоверсионное управление параллельным доступом (MVCC, Multi-Version Concurrency Control)

- множественные версии объекта («снимки», **snapshots**);
- завершённое чтение (read committed):
  - при чтении транзакция получает самую последнюю версию объекта **на момент начала очередной инструкции**;
- изоляция моментальных снимков (SI, snapshot isolation):
  - при чтении транзакция получает самую последнюю версию объекта **на момент начала данной транзакции**;
  - фиксация транзакции происходит только если изменяемые данные не были изменены (зафиксированы) другой транзакцией (“first committer wins”);
- сериализуемая изоляция моментальных снимков (SSI, erializable snapshot isolation):
  - при чтении транзакция получает самую последнюю версию объекта **на момент начала данной транзакции;**
  - фиксация транзакции происходит только если изменяемые данные не были изменены (зафиксированы) или прочитаны другой транзакцией;
- собственные изменения видны в транзакции.

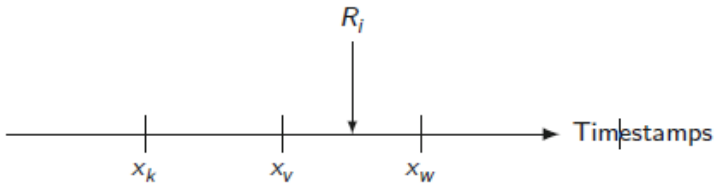
Многоверсионное управление параллельным доступом: сериализуемая изоляция моментальных снимков

- множественные версии объекта («снимки»);
- при чтении транзакция получает самую последнюю версию объекта **на момент начала данной транзакции**;
- при записи:

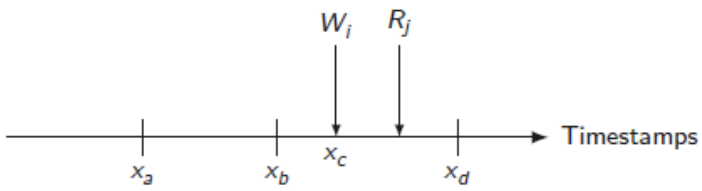
$$TS(T_i) < RTS(X_k) = TS(T_j) \Rightarrow cancel\, T_i$$
$$X_k^t \rightarrow X_k^{t+1}; RTS(X_k^{t+1}) = WTS(X_k^{t+1}) = TS(T_i)$$

- условие удаления «устаревших» версий:

$$\exists X_k^{t+1} \mid TS(X_k^{t+1}) > TS(X_k^t) \;\&\; \forall T_i\; TS(T_i) > TS(X_k^{t+1})$$



(a)



(b)

6. Восстановление баз данных.

- Три типа сбоев в базах данных:
  - сбои транзакций (transaction failures) – откаты;
  - системные сбои (system failures) – потеря содержимого оперативной памяти;
  - сбои носителей (media failures) – потеря данных на диске;
- Для предотвращения катастрофических последствий при сбое носителей:
  - Дублирование дисков;
  - создание резервных копий базы данных (полных или инкрементных копий);
- Восстановление после сбоев транзакций и системных сбоев зависит от организации записи обновлений основных данных в СУБД:
  - обновление с перезаписью (in-place updating);
  - обновление с заменой (out-of-place updating);
- Для восстановления в СУБД с перезаписью необходим журнал базы данных (database log):

  - синонимы: журнал транзакций (transaction log), журналом завершённых транзакций (commit log), журнал повторного выполнения (redo log) или журнал опережающей записи (write-ahead log, WAL);
  - содержит информацию для восстановления (recovery information), которая необходима для выполнения набора идемпотентных операций.
- повторная отработка (reprocessing): периодические снимки базы данных и записи о всех операциях со времени последнего сохранения резервной копии – нереализуемый подход в силу отсутствия времени и асинхронного характера первоначальных изменений;
- откат-накат (rollback-rollforward):
  - периодические снимки базы данных и журнал транзакций со времени последнего сохранения;
  - rollforward: выполнение всех корректных транзакций (для повторного выполнения транзакции журнал должен содержать копию записей базы данных после их изменения – конечные образы, after-images);
  - rollback: отмена изменений, вызванных некорректными или незавершенными транзакциями (для отмены транзакции журнал должен содержать копию записей базы данных до их изменения – исходные образы, before-images);

7. Управление параллельной обработкой в СУБД MS SQL Server.

- определение уровня изоляции транзакции:

```
SET TRANSACTION ISOLATION LEVEL
{ READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ | SNAP
Тема 11. Транзакции 19
SHOT | SERIALIZABLE }
```

- определение характеристик курсора:

```
DECLARE cursor_name CURSOR
[ LOCAL | GLOBAL ]
[ FORWARD_ONLY | SCROLL ]
[ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ] [ READ_ONLY | SCROLL
\_LOCKS | OPTIMISTIC ] [ TYPE_WARNING ]
FOR select_statement
[ FOR UPDATE [ OF column_name [ ,...n ] ] ]
```

- блокировочные подсказки инструкций (SELECT, INSERT, UPDATE, DELETE):

```
SELECT … FROM … WITH
```

8. Транзакции в СУБД MS SQL Server. Режимы транзакций.

- возможные уровни изоляции (блокировка / версионирование):
  - READ UNCOMMITTED;

- **READ COMMITTED;** (по умолчанию)
  - READ_COMMITTED_SNAPSHOT (ON | OFF);
- REPEATABLE READ;
- SNAPSHOT – согласованность на уровне транзакции;
- SERIALIZABLE;

- возможно переключение уровня изоляции внутри транзакции (кроме SNAPSHOT, правила блокировок применяются на основании текущего уровня изоляции);
- при выходе из хранимой процедуры / триггера уровень изоляции восстанавливается до уровня, актуального на момент вызова.

Уровни изоляции транзакций

		Isolation Level			
		Read Uncommitted	Read Committed	Repeatable Read	Serializable
Problem Type	Dirty Read	Possible	Not Possible	Not Possible	Not Possible
	Nonrepeatable Read	Possible	Possible	Not Possible	Not Possible
	Phantom Read	Possible	Possible	Possible	Not Possible

- разделяемые блокировки удерживаются все время до завершения транзакции

Режимы транзакций

- явные транзакции:
  - запускаются посредством инструкции BEGIN TRANSACTION;
  - завершение транзакции осуществляется с использованием инструкций COMMIT TRANSACTION / WORK (фиксация) или ROLLBACK TRANSACTION / WORK (откат);
  - управление транзакциями также может осуществляться через функции API – ADO, ODBC, OLEDB (ITransactionLocal::StartTransaction(), ITransaction::Commit() и ITransaction::Abort());
- автоматически фиксируемые транзакции (режим по умолчанию):
  - каждая отдельная инструкция фиксируется после завершения;
- неявные транзакции:
  - режим запускается через инструкцию SET IMPLICIT_TRANSACTIONS ON или через соответствующие функции API (ODBC, OLEDB);
  - очередная инструкция ALTER TABLE, INSERT, CREATE, OPEN, DELETE, REVOKE, DROP, SELECT, FETCH, TRUNCATE TABLE, GRANT, UPDATE автоматически запускает новую транзакцию;

- необходимо только фиксировать или выполнять откат каждой транзакции (COMMIT / ROLLBACK), после завершения транзакции следующая инструкция (из перечисленного списка) запускает новую транзакцию.

Неявные транзакции: пример (T-SQL)

```
CREATE TABLE ImplicitTran(
  Cola int PRIMARY KEY,
  Colb char(3) NOT NULL);
GO
SET IMPLICIT_TRANSACTIONS ON;

--First implicit transaction started by an INSERT statement. INSERT INTO
ImplicitTran VALUES (1, 'aaa');
INSERT INTO ImplicitTran VALUES (2, 'bbb');

--Commit first transaction.
COMMIT TRANSACTION;

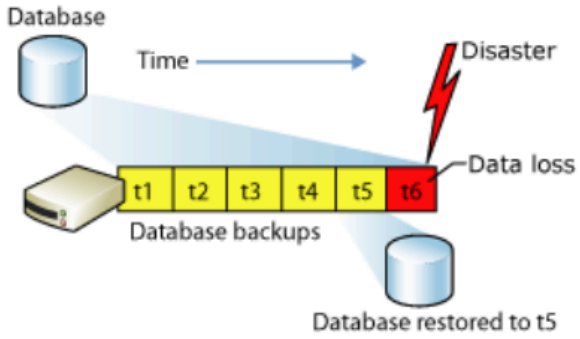
--Second implicit transaction started by a SELECT statement.
SELECT COUNT(*) FROM ImplicitTran;
INSERT INTO ImplicitTran VALUES (3, 'ccc');
SELECT * FROM ImplicitTran;

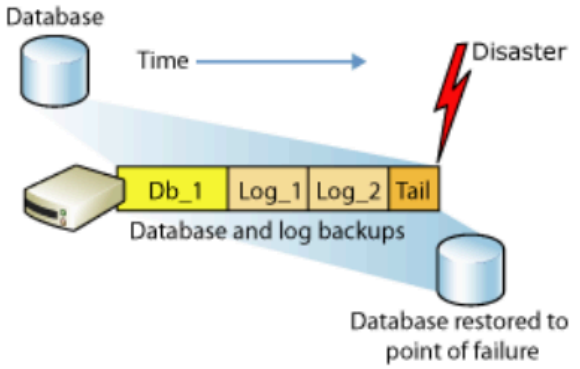
--Commit second transaction.
COMMIT TRANSACTION;
SET IMPLICIT_TRANSACTIONS OFF;
```

9. Восстановление баз данных. Восстановление в СУБД MS SQL Server.

Восстановление

- создание резервной копии (данных и журнала транзакций):
  - полные
  - частичные
- модели восстановления:
  - полная
  - простая
  - частичная





Вопрос 3

1. ADO.NET: режимы работы, объектная модель, базовые интерфейсы и базовые классы. Поставщики данных.

Режимы работы ADO.NET

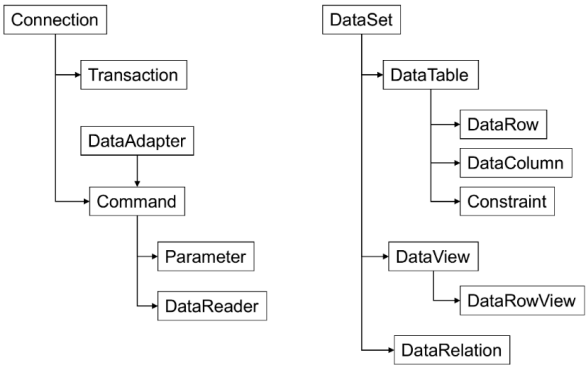
Режим с поддержанием соединения (связный уровень, connected layer):

- осуществляется непосредственное, явное подключение к источнику данных и отключение от него;
- используются объекты Connection, Command, DataReader;

Режим без поддержания соединения (несвязный уровень, disconnected layer):

- операции с данными производятся над локальной копией данных источника, представленной объектом DataSet и связанными объектами (DataTable etc.);
- используется объект DataAdapter для обеспечения взаимодействия приложения и источника данных, при этом операции открытия и закрытия соединения выполняются автоматически по мере необходимости;
- фактически операции с объектами несвязного уровня можно выполнять вообще без подключения к источнику данных.

Объектная модель ADO.NET



Основные объекты поставщиков данных ADO.NET

Объект	Базовый класс	Реализуемые интерфейсы	Назначение
Connection	DbConnection	IDbConnection	Обеспечивает подключение к источнику данных
Command	DbCommand	IDbCommand	Представляет запрос к источнику данных
DataReader	DbDataReader	IDataReader IDataRecord	Обеспечивает однонаправленный просмотр данных (на основе курсора, открытого на сервере)
DataAdapter	DbDataAdapter	IDataAdapter IDbDataAdapter	Обеспечивает обмен данными между источниками данных и объектами наборов данных (DataSet)
Parameter	DbParameter	IDataParameter IDbDataParameter	Представляет именованный параметр в параметризованном запросе
Transaction	DbTransaction	IDbTransaction	Инкапсулирует транзакцию

2. Связный уровень ADO.NET. Объекты связного уровня и общий сценарий работы.

Общий сценарий работы

1. создание, настройка и открытие объекта соединения (Connection);

```
SqlConnection cn = new SqlConnection();
cn.ConnectionString = @"Data
Source=(local)\SQLEXPRESS;Integrated Security=SSPI;
```

2. создание и настройка объекта запроса (Command), в том числе:

- определение объекта соединения для объекта запроса;
- определение строки запроса;

```
Initial Catalog=AutoLot";
cn.Open();
string strSQL = "Select \* From Inventory";
SqlCommand myCommand = new SqlCommand(strSQL,cn);
```

3. вызов метода ExecuteReader() объекта DataReader для получения результатов запроса;

```
SqlDataReader myDataReader;
myDataReader = myCommand.ExecuteReader();
```

4. прокрутка результатов выполнения запроса

```
while (myDataReader.Read())
{ …}
myDataReader.Close();
(DataReader.Read()).
cn.Close()
```

Объекты связного уровня

Основные объекты поставщиков данных ADO.NET

Объект	Базовый класс	Реализуемые интерфейсы	Назначение
Connection	DbConnection	IDbConnection	Обеспечивает подключение к источнику данных
Command	DbCommand	IDbCommand	Представляет запрос к источнику данных
DataReader	DbDataReader	IDataReader IDataRecord	Обеспечивает однонаправленный просмотр данных (на основе курсора, открытого на сервере)
DataAdapter	DbDataAdapter	IDataAdapter IDbDataAdapter	Обеспечивает обмен данными между источниками данных и объектами наборов данных (DataSet)
Parameter	DbParameter	IDataParameter IDbDataParameter	Представляет именованный параметр в параметризованном запросе
Transaction	DbTransaction	IDbTransaction	Инкапсулирует транзакцию

3. Связный уровень ADO.NET. Общий сценарий работы. Пул подключений.

Общий сценарий работы:

1. создание, настройка и открытие объекта соединения (Connection);
2. создание и настройка объекта запроса (Command), в том числе:
   - определение объекта соединения для объекта запроса;
   - определение строки запроса;
3. вызов метода ExecuteReader() объекта DataReader для получения результатов запроса;
4. прокрутка результатов выполнения запроса (DataReader.Read()).

```
SqlConnection cn = new SqlConnection();
cn.ConnectionString = @"Data Source=(local)\SQLEXPRESS;Integrated
Security=SSPI; Initial Catalog=AutoLot";
cn.Open();

string strSQL = "Select * From Inventory";
SqlCommand myCommand = new SqlCommand(strSQL, cn);

SqlDataReader myDataReader;
myDataReader = myCommand.ExecuteReader();
```

```
while (myDataReader.Read())
{ ... }
myDataReader.Close();
cn.Close()
```

Пул соединений:

- пул соединений позволяет улучшить производительность за счет повторного использования установленного физического соединения с источником;
- пул соединений поддерживается поставщиками подключений ADO.NET и по умолчанию используется (Pooling = true);
- в момент соединения поставщик определяет:
  - существует ли пул для данной строки подключения;
  - есть ли в данном пуле свободные (неактивные) соединения;
- минимальный размер пула определяется параметром (MinPoolSize = 0):
  - в момент открытия первого соединения пула поставщик устанавливает необходимое число физических подключений к источнику;
  - поставщик поддерживает минимальное число соединений в пуле независимо от того, являются ли они активными;
- максимальный размер пула определяется параметрами (MaxPoolSize = 100), а в случае, когда достигнут максимальный размер пула и предпринимается попытка открытия нового соединения:
  - производится ожидание освобождения соединения пула в течении ConnectionTimeout = 15;
  - по истечении таймаута возникает исключение InvalidOperationException;
- при закрытии соединения и уничтожении объекта соединения внутреннее соединение с СУБД не разрывается, а помещается в пул для дальнейшего использования;
- соединение в пуле, которое не использовалось, уничтожается по истечении таймаута;
- возможна принудительная очистка пулов путем вызова функций ClearPool() и ClearAllPools().

4. Связный уровень ADO.NET. Поставщик данных. Создание программного кода, не зависящего от поставщика.

```
<?xml version="1.0" encoding="utf-8" ?>
<connectionStrings>
  <add name ="ASClientCS"
    providerName="System.Data.SqlClient"
    connectionString="Server=192.168.0.92\\sqls2005; Database=OneToManyMerge;
        User ID=sa; Password=Hc6X"/>
</connectionStrings>
```

```
// Get the factory provider.
DbProviderFactory df = DbProviderFactories.GetFactory(
  ConfigurationManager.ConnectionStrings["ASClientCS"].ProviderName
);

// Now make connection object.
DbConnection cn = df.CreateConnection();
cn.ConnectionString =
ConfigurationManager.ConnectionStrings["ASClientCS"].ConnectionString;
cn.Open();

// Make command object.
DbCommand cmd = df.CreateCommand();
cmd.Connection = cn;
cmd.CommandText = "Select \* From Inventory";

DbDataReader dr = cmd.ExecuteReader(CommandBehavior.CloseConnection);
while (dr.Read()) {…}
dr.Close();
```

5. Связный уровень ADO.NET. Параметризированные запросы.

Command

- позволяет выполнить:
  - команду языка SQL (CommandType == Text);
  - хранимую процедуру (CommandType == StoredProcedure);
- с точки зрения возвращаемых данных запросы могут быть отнесены к одному из трех типов:
  - не возвращающие результатов [INSERT, UPDATE, DELETE] : ExecuteNonQuery();
  - возвращающие одиночное (скалярное) значение [SELECT COUNT(…)…]: ExecuteScalar();
  - возвращающие набор результатов : ExecuteReader();
- SqlCommand позволяет организовать асинхронный доступ к данным:
  - BeginExecuteReader() / EndExecuteReader();
  - BeginExecuteNonQuery() / EndExecuteNonQuery();
  - BeginExecuteXmlReader() / EndExecuteXmlReader();

```
string connectionString = "Data Source=localhost;" + "Initial
Catalog=Northwind;Integrated Security=SSPI;";
SqlConnection conn = new SqlConnection(connectionString);
conn.Open();

SqlCommand cmd = conn.CreateCommand();
cmd.CommandText = "DELETE FROM Customers WHERE CustomerID = 'ALFKI'";
Console.WriteLine("RecordsAffected" + cmd.ExecuteNonQuery());

cmd = conn.CreateCommand();
```

```
cmd.CommandText = "SELECT COUNT(*) FROM Customers";
Console.WriteLine(Convert.ToInt32(cmd.ExecuteScalar()));

cmd = new SqlCommand("SELECT CustomerID, CompanyName FROM Customers", conn);
SqlDataReader rdr = cmd.ExecuteReader();
while (rdr.Read()) {
  Console.WriteLine(
    rdr["CustomerID"] + " - " + rdr["CompanyName"]);
}
rdr.Close();

conn.Close();
```

Параметризированные запросы: Parameter

```
string sql = string.Format("Insert Into Inventory" + "(CarID, Make, Color)
Values (@CarID, @Make, @Color)");

// This command will have internal parameters.
using(SqlCommand cmd = new SqlCommand(sql, this.sqlCn)) {
  // Fill params collection.
  SqlParameter param = new SqlParameter();
  param.ParameterName = "@CarID";
  param.Value = id;
  param.SqlDbType = SqlDbType.Int;
  cmd.Parameters.Add(param);

  param = new SqlParameter();
  param.ParameterName = "@Make";
  param.Value = make;
  param.SqlDbType = SqlDbType.Char;
  param.Size = 10;
  cmd.Parameters.Add(param);

  param = new SqlParameter();
  param.ParameterName = "@Color";
  param.Value = color;
  param.SqlDbType = SqlDbType.Char;
  param.Size = 10;
  cmd.Parameters.Add(param);

  cmd.ExecuteNonQuery();
}
```

SqlParameter принимает следующие основные поля:

- `SqlDbType` — тип требуемых данных;
- `Direction` — одно из значений `Input`, `InputOutput`, `Output`, `ReturnValue` — т.е., параметр это только ввод, ввод/вывод, только вывод или является результатом вызова хранимой процедуры, встроенной функции или заданной пользователем функции;

- `IsNullable` — принимает ли параметр `null` значение. Не проверяет данных, поэтому не защищает от отправки/получения `null`;
- `ParameterName` — имя параметра;
- `Size` — максимальный размер столбца в байтах;
- `Value` — значение параметра.

Выполнение хранимых процедур

```
using (SqlCommand cmd = new SqlCommand("GetPetName", this.sqlCn)) {
  cmd.CommandType = CommandType.StoredProcedure;

  SqlParameter param = new SqlParameter();
  param.ParameterName = "@carID";
  param.SqlDbType = SqlDbType.Int;
  param.Value = carID;
  param.Direction = ParameterDirection.Input;
  cmd.Parameters.Add(param);

  param = new SqlParameter();
  param.ParameterName = "@petName";
  param.SqlDbType = SqlDbType.Char;
  param.Size = 10;
  param.Direction = ParameterDirection.Output;
  cmd.Parameters.Add(param);

  cmd.ExecuteNonQuery();

  Name = (string)cmd.Parameters["@petName"].Value;
}
```

```
CREATE PROCEDURE GetPetName
  @carID int,
  @petName char(10) output
AS
  SELECT @petName = PetName from Inventory
  where CarID = @carID
```

6. Несвязный уровень ADO.NET. Объекты несвязного уровня и общий сценарий работы.

Несвязный уровень ADO.NET

- автономно управляет подключением к источнику данных;
- получает результаты запроса и записывает их в DataSet;
- переносит в БД изменения, внесенные в DataSet;
- устанавливает соответствие между результатами запроса и таблицами / столбцами в DataSet.

DataSet

- содержит данные в виде набора записей, состоящих из одинаковых наборов полей;

- каждая запись хранится как в исходном варианте, так и в текущем с учетом внесенных изменений.

DataTable

- содержит данные в виде набора записей, состоящих из одинаковых наборов полей;
- каждая запись хранится как в исходном варианте, так и в текущем - с учетом внесенных изменений.

DataColumn

- определяет столбец в таблице и его свойства: тип, значение по умолчанию, может ли содержать NULL, может ли значение быть изменено;
- задает выражение для вычисляемых столбцов.

DataRow

- соответствует одной записи в таблице;
- может быть в одном из состояний:
  - Unchanged;
  - Detached;
  - Added;
  - Modified;
  - Deleted;
- содержит следующие версии значений полей:
  - Original;
  - Current;
  - Proposed;
  - Default.

Constraint

- Primary Key – указывает столбцы, входящие в первичный ключ;
- Unique – указывает столбцы, значения которых должно быть уникально в таблице;
- Foreign Key – указывает столбцы, входящие во внешний ключ, а также задает правила поведения ограничений ссылочной целостности (Cascade, SetNull, SetDefault, None).

DataView

- отбирает записи, удовлетворяющие заданному условию;
- отбирает записи, находящиеся в определенном состоянии;
- сортирует записи по значениям указанных столбцов;
- может основываться только на одной таблице.

DataRelation

- устанавливает связь один-к одному, один-ко-многим или многие-ко-многим между двумя столбцами или наборами столбцов;
- по умолчанию, для родительских столбцов автоматически добавляется ограничение UniqueConstraint, а для дочерних – ForeignKeyConstraint;
- позволяет определять родительские и дочерние записи для указанной записи:
  - DataRow.GetParentRow();
  - DataRow.GetParentRows();
  - DataRow.GetChildRows().

7. Несвязный уровень ADO.NET. Настройка запросов для синхронизации с источником данных.

Несвязный уровень ADO.NET

- автономно управляет подключением к источнику данных;
- получает результаты запроса и записывает их в DataSet;
- переносит в БД изменения, внесенные в DataSet;
- устанавливает соответствие между результатами запроса и таблицами / столбцами в DataSet.

Настройка запросов DataAdapter

```
SqlCommand cmd = new SqlCommand(
  "UPDATE [Order Details] " +
  "SET OrderID = ?, ProductID = ?, " +
  "Quantity = ?, UnitPrice = ? “ +
  "WHERE OrderID = ? AND ProductID = ? AND " +
  "Quantity = ? AND UnitPrice = ?", conn);

SqlParameterCollection pc = cmd.Parameters;
pc.Add("OrderID_New", OleDbType.Integer, 0, "OrderID");
pc.Add("ProductID_New", OleDbType.Integer, 0, "ProductID");
pc.Add("Quantity_New", OleDbType.SmallInt, 0, "Quantity");
pc.Add("UnitPrice_New", OleDbType.Currency, 0, "UnitPrice");

SqlParameter param;
param = pc.Add("OrderID_Orig", OleDbType.Integer, 0, "OrderID");
param.SourceVersion = DataRowVersion.Original;
param = pc.Add("ProductID_Orig", OleDbType.Integer, 0, "ProductID");
param.SourceVersion = DataRowVersion.Original;
param = pc.Add("Quantity_Orig", OleDbType.SmallInt, 0, "Quantity");
param.SourceVersion = DataRowVersion.Original;
param = pc.Add("UnitPrice_Orig", OleDbType.Currency, 0, "UnitPrice");
param.SourceVersion = DataRowVersion.Original;
dataAdapter.UpdateCommand = cmd;
```

Вопрос 4

1. Распределение данных в СУБД. Горизонтальная и вертикальная фрагментация.

Горизонтальная фрагментация выполняется операцией селекции, распределяющей кортежи в соответствии с предикатом селекции (CHECK в столбцах ID); таблица разбивается на несколько частей так, чтобы каждая часть содержала записи со значениями только из некоторого диапазона;

Создание горизонтальной фрагментации:

- на каждом сервере: CREATE TABLE Table (ID INTEGER PRIMARY KEY CHECK (ID < 1000), ColumnA INTEGER, ColumnB INTEGER)
- на сервере, содержащем представление: USE master EXEC sp_serveroption OtherServer', 'lazy schema validation', 'true' CREATE VIEW View AS SELECT **FROM MyDB.dbo.Table UNION ALL SELECT** FROM OtherServer.MyDB.dbo.Table

Вертикальная фрагментация реализуется с применением операции проекции, необходимы дополнительные соединения для достижения значений нерезидентных атрибутов; таблица разбивается на части так, чтобы каждая часть содержала только некоторые из столбцов исходной таблицы (разбиение строк)

Создание вертикальной фрагментации:

- на первом сервере: CREATE TABLE Table (ID INTEGER PRIMARY KEY, ColumnA INTEGER)
- на втором сервере: CREATE TABLE Table (ID INTEGER PRIMARY KEY, ColumnB INTEGER)
- на сервере, содержащем представление: CREATE VIEW View AS SELECT one.ID, one.ColumnA, two.ColumnB FROM MyDB.dbo.Table one, OtherServer.MyDB.dbo.Table two WHERE one.ID = two.ID

2. Распределение данных в СУБД: репликация. Согласованность реплик и распространение изменений. Репликация в СУБД SQL Server.

Репликация – создание множественных копий данных (таблиц);

- процедура репликации должна обладать прозрачностью для клиентов СУБД;
- согласованность реплик:
  - строгая (глобальная) согласованность: единая логическая копия;
  - согласованность на уровне отдельных транзакций;
  - локальная согласованность (например, разделение реплик для оперативной и аналитической обработки);
- согласованность реплик определяют правила обновления и распространения изменений по репликам.

Репликация главной копии

- различные методы распространения обновлений, влияющие на:
  - доступность системы;
  - отставание состояния резервных серверов;
  - связанную с репликацией нагрузку на серверы;
- синхронное распространение изменений: до фиксации транзакции, выполнившей изменения на главном сервере;
- асинхронное распространение изменений, предполагающее отставание состояния резервных серверов:
  - (повторение операторов SQL);
  - обновление логических записей;
  - на уровне страниц баз данных;
  - распространение записей журнала.

Варианты использования

- создание копий, отражающих состояние базы данных на основном сервере на определённый момент времени (без последующего распространения изменений);
- перераспределение нагрузки на главный сервер (перераспределение операций чтения на резервные серверы);
- перераспределение запросов в зависимости от типов нагрузки (оперативная и аналитическая обработка данных);
- создание резервных копий базы данных для переключения в случае отказа.

СУБД SQL Server поддерживает следующие механизмы репликации:

- транзакционная репликация (transactional replication) начинается с создания исходного моментального снимка объектов и данных базы данных публикации, последующие изменения данных и схемы на издателе обычно доставляются подписчику без задержек (практически в реальном времени), а изменения данных применяются на подписчике в том же порядке и в тех же рамках транзакций, в которых они выполнялись у издателя (в пределах публикации гарантируется согласованность транзакций);
- репликация слиянием (merge replication);
- репликация моментальных снимков (snapshot replication) распространяет данные подписчикам целиком и точно в том виде, в котором они были представлены в момент синхронизации, и не контролирует обновления этих данных.

3. Параллелизм в СУБД. Параллельные СУБД.

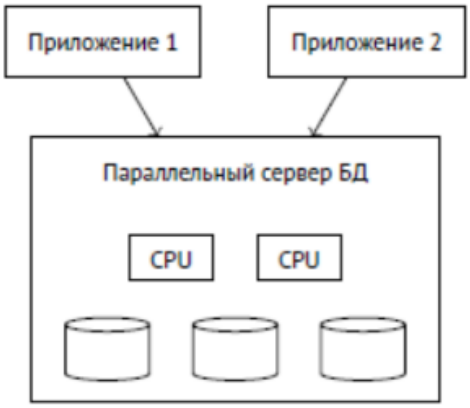
Параллелизм

- гранулярность параллелизма:

  - между транзакциями;
  - между запросами внутри транзакций;
  - между операциями плана выполнения внутри запросов;
  - в алгоритмах выполнения операций;
- издержки:
  - синхронизация между параллельно выполняемыми операциями с данными;
  - проблема равномерного распределения нагрузки;
- преимущества: высокая масштабируемость для отдельных типов операций.

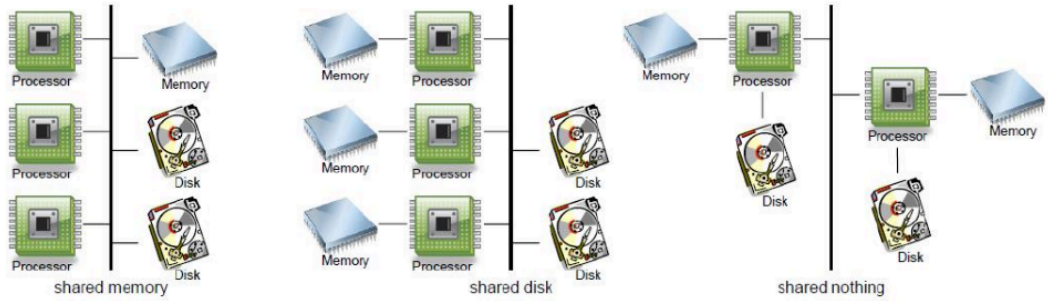
Параллельные СУБД

- основная цель: обеспечение высокой производительности за счет выполнения запроса (транзакции) или отдельных операций на нескольких вычислительных узлах (устройствах);
- единая схема данных (для каждой базы данных);
- проблема балансировки нагрузок между вычислительными узлами как с точки зрения выполнения запросов, так и с точки зрения распределения хранимых данных (для архитектуры SN из-за неравномерности скорости доступа к носителям данных).
- с точки зрения клиента не отличается от централизованной СУБД;



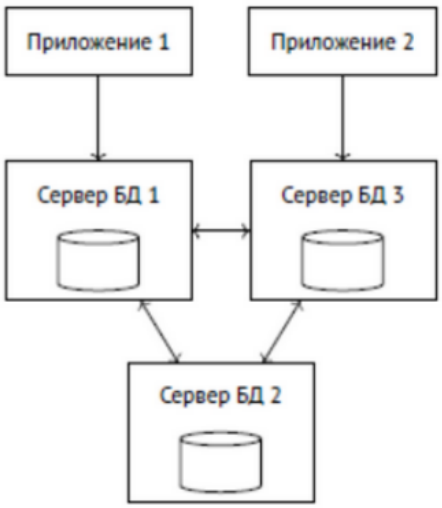
Аппаратная конфигурация параллельных СУБД

- SM (shared memory):
  - многопроцессорные системы с разделяемой памятью;
  - масштабируемость ограничена возможным количеством устанавливаемых процессоров в одной вычислительной системе и пропускной способностью;
- SD (shared disk):
  - вычислительные устройства взаимодействуют между собой и с устройствами дисковой памяти через коммуникационную сеть;
  - масштабируемость ограничена необходимостью синхронизации для предотвращения конфликтующих изменений страниц;
- SN (shared nothing):
  - каждая вычислительная система функционирует автономно и взаимодействие через вычислительную сеть.



4. Распределённые СУБД.

- основные цели:
  - повышение доступности;
  - горизонтальная масштабируемость;
  - интеграция разнородных систем;
- совокупность серверов баз данных, каждый из которых может работать независимо от остальных серверов;
- отдельный сервер поддерживает свою схему данных, но может выполнять запросы к данным, размещённым на нескольких серверах (прозрачный доступ к данным);
- с точки зрения используемых на серверах СУБД могут быть однородными и неоднородными;
- возможные ограничения реализации:
  - доступность некоторых данных;
  - выполнение требований ACID для распределённых транзакций.



5. Согласованность в распределённых системах. Протокол двухфазной фиксации транзакций. Распределённые транзакции в СУБД SQL Server.

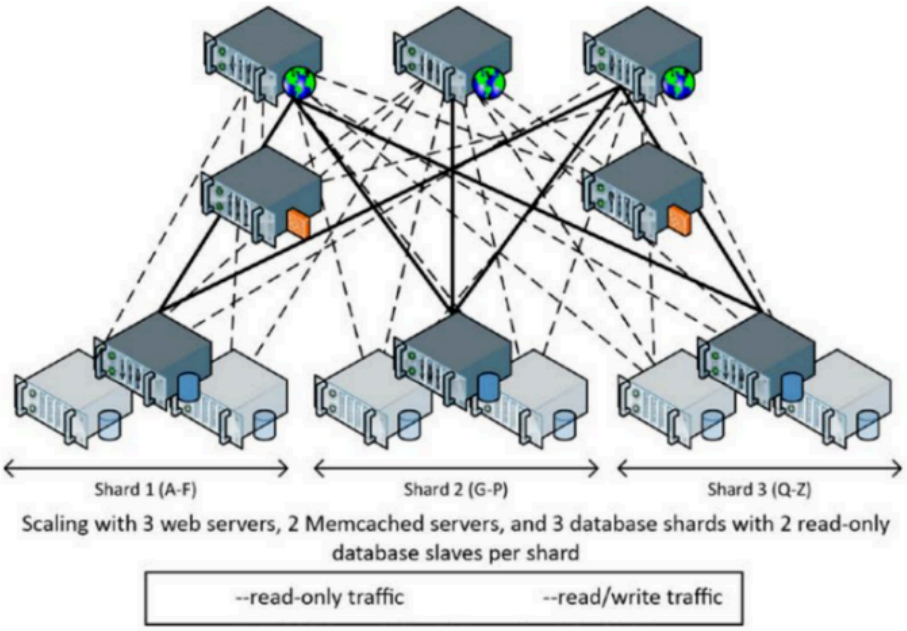
- глобальные (распределённые) транзакции – содержат операции обработки элементов данных на разных серверах:

  - должны удовлетворять требуемому уровню изоляции;
  - должны завершаться одинаково (фиксацией или откатом);
- локальные транзакции – обрабатывают данные, находящиеся на одном сервере;
- глобальные протоколы управления транзакциями должны обеспечивать высокую доступность, отсутствие централизованных механизмов управления и горизонтальную масштабируемость → ослабленные критерии согласованности в реализациях распределённых систем;
- протоколы на основе меток времени: распределённый протокол изоляции моментальных снимков.

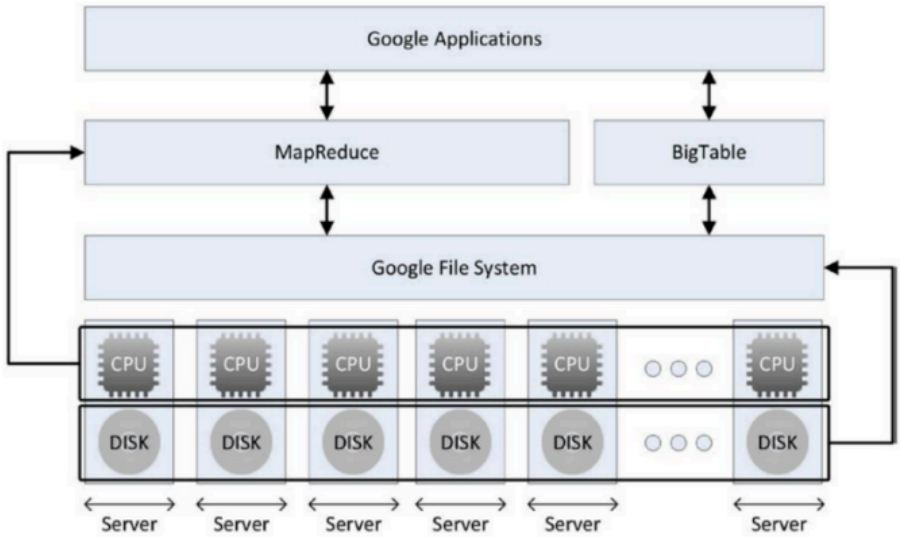
Двухфазная фиксация транзакций

- протокол двухфазной фиксации транзакций (**2PC – two-phase commit protocol**) определяет алгоритм завершения распределённых транзакций
- участники распределенной транзакции:
  - два или более сервера, которые называются **диспетчерами ресурсов**;
  - **диспетчер транзакций**, который обеспечивает управление транзакцией путем координации между диспетчерами ресурсов;
- двухфазная фиксация транзакций:
  - фаза подготовки: каждый из диспетчеров ресурсов обеспечивает устойчивость транзакции, в частности, записывая журнал транзакции на диск;
  - фаза фиксации: начинается в случае успешного завершения подготовки всеми диспетчерами ресурсов и завершается после получения диспетчером транзакций подтверждений об успешной фиксации от каждого из диспетчеров ресурсов;

Пример: сегментирование данных в web-системах



Пример: программно-аппаратная архитектура Google (Google BigTable / HBase)



Распределённые транзакции в СУБД SQL Server

- SQL Server обеспечивает выполнение и управление удалёнными и распределёнными транзакциями в том случае, если транзакции поддерживаются источником данных и поставщиком (в противном случае операции чтения/записи могут быть запрещены или ограничены);
- распределённая транзакция инициируется явно (инструкцией BEGIN DISTRIBUTED TRANSACTION) или неявно (выполнением распределённого запроса или удалённой хранимой процедуры внутри локальной транзакции);
- процедура завершения распределённой транзакции выполняется в соответствии с протоколом двухфазной фиксации, причём в качестве диспетчера транзакций может выступать координатор распределенных транзакций MS DTC (Distributed Transaction Coordinator) или любой другой диспетчер транзакций, поддерживающий спецификацию X/Open XA.

6. Горизонтальные секционирование в СУБД SQL Server. Проектирование и создание распределённых секционированных представлений.

Горизонтальное секционирование

- распределённое секционированное представление формируется как набор строк из отдельных таблиц-элементов, объединённых с помощью UNION ALL.
- ограничения для базовых таблиц:
  - не должны входить в запрос более одного раза;
  - не могут иметь индексов, созданных на любых вычисляемых столбцах;
  - все ограничения первичного ключа таблиц-элементов должны быть на одинаковом количестве столбцов;

  - должна быть одна и та же настройка дополнения ANSI (параметр конфигурации ANSI_PADDING);
- ограничения для списка выборки:
  - все столбцы в каждой таблице-элементе должны быть включены в список выборки;
  - на столбцы нельзя ссылаться в списке выборки больше одного раза;
  - столбцы должны быть одного типа, идти в одном и том же порядке и встречаться только один раз.

Проектирование распределенных секционированных представлений

- при проектировании набора распределенных секционированных представлений необходимо выполнить следующие процедуры:
  - определить шаблон инструкций SQL, выполняемых приложением;
  - определить, каким образом таблицы связаны друг с другом (через внешние ключи или неявно через столбцы, используемые в соединениях);
  - сопоставить частоту инструкций SQL с секциями, определенными при анализе связей;
  - определить правила маршрутизации инструкций SQL;
- цель проектирования: локальное размещение большей части данных, необходимых для выполнения запросов, обрабатываемых конкретным сервером (хотя итоговый набор секций и таблиц может быть асимметричным);
- пример: секционирование данных по регионам.

Ограничения для столбца секционирования

- для секционирования может использоваться только один столбец (столбец секционирования – **partitioning column**), который должен присутствовать в каждой таблице-элементе и находиться в одинаковом порядковом расположении в списках выборки всех инструкций SELECT в представлении (возможно использование SELECT *);
- столбец секционирования:
  - должен входить в первичный ключ;
  - не должен быть определён как вычисляемый, с автоматическим приращением, со значением по умолчанию или типа timestamp;
  - должен иметь единственное ограничение CHECK с операторами [IN, BETWEEN, AND, OR, <, <=, >, >=, =];
- диапазоны значений ключа секционирования, определённые во всех таблицах, не должны пересекаться.

Создание распределенных секционированных представлений: пример

```
--On Server1:
CREATE TABLE Customers_33
(CustomerID INTEGER PRIMARY KEY
CHECK (CustomerID BETWEEN 1 AND 32999), ... -- Additional column definitions)

--On Server2:
```

```
CREATE TABLE Customers_66
(CustomerID INTEGER PRIMARY KEY
CHECK (CustomerID BETWEEN 33000 AND 65999), ... -- Additional column
definitions)

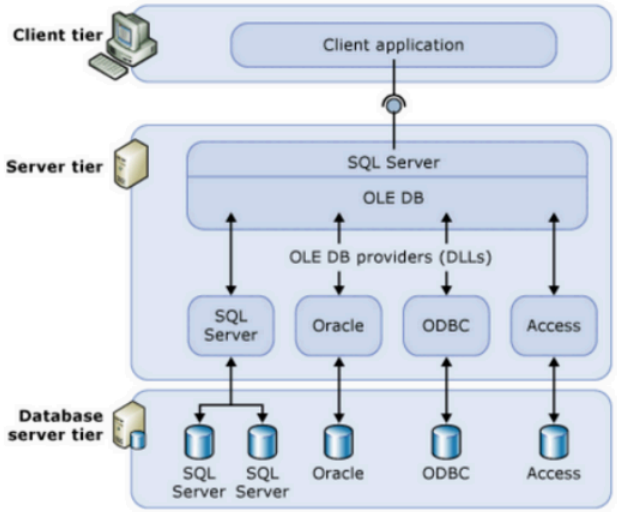
--On Server3:
CREATE TABLE Customers_99
(CustomerID INTEGER PRIMARY KEY
CHECK (CustomerID BETWEEN 66000 AND 99999), ... -- Additional column
definitions)

--On EACH server:
CREATE VIEW Customers AS
SELECT * FROM CompanyDatabase.TableOwner.Customers_33
UNION ALL
SELECT * FROM Server2.CompanyDatabase.TableOwner.Customers_66
UNION ALL
SELECT * FROM Server3.CompanyDatabase.TableOwner.Customers_99
```

7. Распределённые запросы. Взаимодействие с поставщиками данных. Выполнение и оптимизация распределённых запросов.

Распределенные запросы

- распределенные запросы используются для доступа к данным, внешним по отношению к экземпляру СУБД SQL Server и расположенным в возможно гетерогенных источниках данных:
  - в других экземплярах SQL Server;
  - в различных реляционных и нереляционных источниках данных, представляющих данные в табличных объектах, именуемых наборами строк (rowset);
- доступ к внешним данным осуществляется посредством поставщиков данных OLE DB (OLEDB providers).



Взаимодействие с поставщиками OLE DB

- поставщики OLE DB для различных источников данных могут в зависимости от реализации поддерживать следующие функции:
  - только открытие и чтение набора строк базовой таблицы;
  - выполнение инструкций SELECT, INSERT, UPDATE, DELETE;
  - выполнение транзакций;
  - сканирование индексов;
  - использование курсоров, в том числе для изменения и удаления данных;
  - использование параметризованных запросов;
- при обработке распределённых запросов экземпляр SQL Server также выполняет следующие действия:
  - подключение к источнику данных;
  - получение необходимых метаданных;
  - управление выполнением транзакций;
  - преобразование типов данных (между типами OLE DB и встроенными типами);
  - обработку ошибок;
  - управление правами доступа.

Выполнение и оптимизация распределённых запросов

- SQL Server максимально делегирует выполнение распределенного запроса поставщику данных в зависимости от поддерживаемых им возможностей;
- на степень делегирования в том числе влияют:
  - уровень диалекта (грамматики SQL-запросов – SQL Minimum, ODBC Core, SQL-92 Entry level, SQL Server) и дополнительные возможности синтаксиса запросов, поддерживаемые поставщиком;
  - совместимость параметров сопоставления строк (**collation**);
- при оптимизации плана выполнения запроса при наличии поддержки со стороны поставщика данных также могут использоваться:
  - сканирование индексов удалённых таблиц;
  - использование статистик удалённых таблиц;
  - выполнение параметризованных запросов;
  - информация об ограничениях, определённых в базовых таблицах.

8. Связанные серверы. Нерегистрируемые серверы (OPENDATASOURCE, OPENROWSET). Сквозные запросы (OPENQUERY).

Связанным сервером (linked server) называется зарегистрированный на экземпляре СУБД SQL Server источник данных с указанием соответствующего поставщика OLE DB и параметров подключения;

- в инструкциях языка SQL (SELECT, INSERT, UPDATE, DELETE) к объектам связанного сервера можно обращаться путём указания их полного

имени (< **linked-server** >.<**catalog**>. <**schema**>.<**object**>), включающего имя связанного сервера;
- параметры **catalog**, **schema** и **object_name** представляют собой ссылку на объект источника данных, представляемый как набор строк, и передаются поставщику OLE DB для идентификации конкретного объекта данных (для СУБД SQL Server параметр **catalog** соответствует базе данных);
- доступ к связанному серверу также может осуществляться следующими способами:
  - через команду OPENQUERY для выполнения сквозного запроса (**pass-through query**) на связанном сервере;
  - через инструкцию EXECUTE AT **linked_server_name**, позволяющую выполнить произвольный динамически сформированный параметризованный запрос на удалённом сервере.

Операции со связанными серверами

- получение списка зарегистрированных серверов:
  - sp_linkedservers;
  - sys.servers (представление системного каталога);
- добавление связанного сервера:
  - sp_addlinkedserver;
- удаление связанного сервера:

– sp_dropserver;
- добавление / удаление имени входа (сопоставление имени и пароля удалённого входа для заданного локального входа):
  - sp_addlinkedsrvlogin
  - sp_droplinkedsrvlogin

Нерегистрируемые серверы (специальные серверы, ad hoc servers)

- команды OPENROWSET и OPENDATASOURCE обеспечивают возможность доступа к удалённым источникам данных без регистрации связанного сервера;
- команда OPENDATASOURCE:
  - может использоваться там, где синтаксис Transact-SQL позволяет указать имя связанного сервера;
  - требует указания параметров подключения к источнику данных OLE DB (поставщика данных и строки подключения);
- команда OPENROWSET:
  - может применяться везде, где в инструкции языка Transact-SQL используется ссылка на таблицу или представление;
  - требует указания параметров подключения к источнику данных OLE DB (поставщика данных и строки подключения), а также имени объекта или запроса, которые должны формировать набор строк;
- команды OPENROWSET и OPENDATASOURCE не поддерживают передачу переменных языка Transact-SQL в качестве аргументов;

- команды OPENROWSET и OPENDATASOURCE рекомендуется использовать только для единичных обращений к источникам данных, так как они не предоставляют всех возможностей определений связанных серверов, а также при каждом использовании требуют указания всей информации, необходимой для подключения (в том числе паролей).

Нерегистрируемые серверы: пример

```
SELECT GroupName, Name, DepartmentID
FROM OPENDATASOURCE('MSOLEDBSQL',
'Server=Seattle1;Database=AdventureWorks2022;TrustServerCertificate=
Yes;Trusted_Connection=Yes;').HumanResources.Department
ORDER BY GroupName, Name;
SELECT d.* FROM OPENROWSET(
'MSOLEDBSQL',
'Server=Seattle1;Trusted_Connection=yes;',
AdventureWorks2022.HumanResources.Department ) AS d;
SELECT a.*
FROM OPENROWSET(
'MSOLEDBSQL', 'Server=Seattle1;Trusted_Connection=yes;', 'SELECT GroupName,
Name, DepartmentID
FROM AdventureWorks2022.HumanResources.Department ORDER BY GroupName, Name'
) AS a;
```

Сквозные запросы

- команда OPENQUERY позволяет выполнить передаваемый запрос к указанному связанному серверу:
  - на команду OPENQUERY как на имя таблицы можно ссылаться из предложения FROM инструкции SELECT, а также в инструкциях INSERT, UPDATE или DELETE;
  - в качестве аргументов нельзя использовать переменные;
  - команда OPENQUERY не может быть использована для выполнения расширенных хранимых процедур на связанном сервере.

```
SELECT * FROM OPENQUERY (OracleSvr,
'SELECT name FROM joe.titles WHERE name = ''NewTitle''');
INSERT OPENQUERY (OracleSvr, 'SELECT name FROM joe.titles')
VALUES ('NewTitle');
UPDATE OPENQUERY (OracleSvr, 'SELECT name FROM joe.titles WHERE id = 101')
SET name = 'ADifferentName';
DELETE OPENQUERY (OracleSvr,
'SELECT name FROM joe.titles WHERE name = ''NewTitle''');.
```